
Introduzione IA

Corso A

Autore

Giuseppe Acocella

2025/26

<https://github.com/Peenguino>

Indice dei Contenuti

1	PARTE I: FORMULAZIONE PROBLEMI SU GRAFI E ALGORITMI INFORMATI E NON INFORMATI	1
2	— — — — — Lezione 1 - Introduzione al Corso - 05/02/2026	1
3	— — — — — Lezione 2 - Agenti - 05/02/2026	1
3.1	Agenti ed Ambiente	1
3.1.1	Razionalità degli Agenti	1
3.1.2	Natura degli Ambienti	1
3.1.3	Struttura di Agenti	2
3.1.4	Tipi di Rappresentazioni di Ambienti	4
3.2	Risolvere Problemi con la Ricerca	4
3.2.1	Definizione Problemi di Ricerca e Soluzioni	4
4	— — — — — Lezione 3 - Algoritmi di Ricerca e Strategie Ricerca non Informata - 10/02/2026	5
4.1	Algoritmo di Ricerca	5
4.1.1	Strategia Best-First	6
4.1.2	Strutture Dati per la Ricerca	6
4.1.3	Concetti di Stato Ripetuto, Ciclo, Cammini Ridondanti	7
4.2	Misure di Prestazione: Valutazione di Bontà Algoritmo	8
4.2.1	Misurazione della Complessità	8
4.3	Algoritmo Ricerca in Ampiezza	8
4.3.1	Analisi Misure Prestazione Ricerca in Ampiezza	9
4.4	Algoritmo di Dijkstra (a Costo Uniforme)	9
4.4.1	Analisi Misure Prestazione Ricerca a Costo Uniforme	9
4.5	Algoritmo Ricerca in Profondità	9
4.5.1	Analisi Misure Prestazione Ricerca in Profondità	9
4.6	Algoritmo Ricerca in Profondità Limitata	10
4.6.1	Analisi Misure Prestazione Ricerca in Profondità Limitata	10
4.7	Algoritmo Ricerca ad Approfondimento Iterativo	10
4.7.1	Analisi Misure Prestazione Ricerca ad Approfondimento Iterativo	10
4.8	Direzione della Ricerca	11
4.8.1	Ricerca Bidirezionale	11
4.8.2	Analisi Misure Prestazione Ricerca Bidirezionale	11
4.9	Confronto Complessivo Misure Prestazione degli Algoritmi	11
5	— — — — — Lezione 4 - Strategie di Ricerca Informata o Euristica - 12/02/2026	11
5.1	Ricerca Best-First Greedy	12
5.1.1	Analisi di Best First Greedy:	12
5.2	Ricerca A*	12
5.2.1	Analisi A*	12
5.2.2	Ammissibilità di Euristica	12
5.2.3	Dimostrazione: Se $h()$ ammissibile allora A* è ottimo	12
5.2.4	Euristica Consistente	13
5.2.5	Euristica Inconsistente	13
5.2.6	Algoritmo A* con Euristica Consistente, Inconsistente, Inammissibile	13

5.2.7	Confini della Ricerca A e Caratteristiche A	14
5.2.8	A* come Algoritmo Ottimamente Efficiente	14
5.2.9	Ricerca Soddisfacente	14
5.2.10	Ricerca A* Pesata	14
5.2.11	Varianti A* per Limitazioni	14
5.3	Funzioni Euristiche	15
5.3.1	Dominazione di Euristiche	15
5.4	Costruzione di Euristiche	15
5.4.1	Euristica da Problema Rilassato	15
5.4.2	Massimizzazione di Insieme di Euristiche Ammissibili	16
5.4.3	Euristica da Sottoproblema	16
5.4.4	Database di Pattern Soluzioni	16
6	— — — — — Lezione 5 - Ricerca Locale e Cenni di Ricerca Online - 17/02/2026	17
6.1	Ricerca Locale	17
6.1.1	Problemi di Ottimizzazione (Hill Climbing, Simulated Annealing, Local Beam Search)	17
6.2	Ricerca Locale in Spazi Continui	20
6.2.1	Utilizzo Hill Climbing in Spazi Continui con Gradiente	21
6.3	Ricerca con Azioni non Deterministiche	21
7	PARTE II: LOGICA E AGENTI	21
8	— — — — — Lezione 6 - Agenti Basati su Conoscenza - 05/03/2026	21
8.1	Agenti Knowledge Based KB e Dichiarativo vs Procedurale	21
8.2	Conseguenza Logica e Knowledge Base (KB)	22
8.3	Logica	23
8.4	Agenti Logici: Calcolo e Logica Proposizionale	23
8.4.1	Definizione Conseguenza Logica	23
8.4.2	Definizione Model Checking	24
8.5	Calcolo Proposizionale negli Agenti Logici	24
8.5.1	Model Checking e Algoritmo TV-Consegue	25
8.5.2	Algoritmi per la Soddisfacibilità (SAT)	25
8.5.2.1	Trasformazione in Forma Normale Congiuntiva (CNF)	25
8.5.2.2	Algoritmo DPLL	25
8.5.2.3	Metodi Locali per SAT - WalkSat	26
9	— — — — — Lezione 7 - Calcolo Proposizionale (PROP) - 06/03/2026	27
9.1	Inferenza come deduzione	27
9.1.1	Correttezza e Completezza	27
9.1.2	Dimostrazione come Ricerca (Direzione, Strategia)	27
10	— — — — — Lezione 8 - Logica del Prim'Ordine (FOL) - 10/03/2026	28
10.1	Sintassi della FOL	28
10.2	Semantica della FOL	29
10.2.1	Semantica degli Operatori di Quantificazione	29
10.3	Interazione con Knowledge Base KB in FOL	29
10.4	Inferenza nella FOL	29
10.4.1	Istanziamento Universale/Esistenziale	30
10.4.2	Grounding e Teorema di Herbrand	30

10.4.3	Forma a Clausole della FOL	30
10.4.4	Trasformazione in FC della FOL	30
10.4.5	Unificazione ed Algoritmo di Unificazione	31
10.5	Inferenza e Completezza del Metodo di Risoluzione	31
10.6	Cenni di Sistemi a Regole	32
10.6.1	Clausole di Horn	32
10.6.2	Backward/Forward Chaining	32
10.6.3	Programmazione Logica	32
10.6.3.1	Risoluzione Selection Linear Definite-clauses (SLD)	32
11	PARTE III: MACHINE LEARNING	33
12	— — — — — Lezione 9 - Introduzione al Machine Learning - 12/03/2026	33
12.1	Overview di un ML	33
12.2	Supervised Learning e Classification/Regression	33
12.3	Unsupervised Learning	33
12.4	Modello	34
12.4.1	Tipi di Modelli - (Lineari, Simbolici, Probabilistici, Instance Based)	34
12.5	Algoritmo di Learning	34
12.6	Generalizzazione del ML	34
13	— — — — — Lezione 10 - Concept Learning - 17/03/2026	35
13.1	Regole Congiuntive	35
13.2	Rappresentazione di Ipotesi	35
13.2.1	Definizione di Prototypical Concept Learning Task	35
13.2.2	Assunzione sulla Inductive Learning Hypothesis	35
13.2.3	Algoritmo Find-S	36
13.2.4	Definizione di Version Spaces	36
13.2.5	Version Space tramite General/Specific Boundary (con Th.)	36
13.2.6	Algoritmo Candidate Elimination	37
13.3	Inductive Bias	37
13.3.1	Definizione Formale	37
13.3.2	Sistemi Induttivi ed Equivalenti Deduttivi e Riassunto Learners	38
14	— — — — — Lezione 11 - Modelli Lineari I - 19/03/2026	38
14.1	Regressione	38
14.1.1	Regressione Lineare Univariata e Metodo Least Mean Square (LMS)	38
14.1.2	Approccio Iterativo via Ricerca Locale	39
14.1.3	Estensione Pattern e Dimensioni Algoritmo a Gradiente Discendente	40
14.2	Linear Basis Expansion (LBE)	41
15	— — — — — Lezione 12 - Modelli Lineari II - 24/03/2026	42
15.1	Underfitting/Overfitting e Regularizzazione	42
15.1.1	Regularizzazione Ridge/Tikhonov	42
15.2	Classificazione per Regressione	43
15.2.1	Decision Boundary	43
15.2.2	Decision Boundary Threshold Classifier	44
15.2.3	Formalizzazione Learning Problem for Linear Classifiers	44
16	— — — — — Lezione 13 - Decision Trees - 26/03/2026	45
16.1	Algoritmo ID3	45

16.1.1	Definizione Entropia	45
16.1.2	Definizione Gain	45
16.2	Inductive Bias del Learning su Decision Tree (DT)	47
16.3	Bias di Ricerca vs Bias di Linguaggio in Modelli Discreti/Lineari	47
16.4	Definizione di Overfitting	47
16.4.1	Strategie di Mitigazione Overfitting (Early Stopping, Pruning)	47
16.5	Gestione Valori Continui, Gestione Valori Mancanti e Gestione Attributi a Costi Non Omogenei	48
17	— — — — — Lezione 14 - Validation and Theoretical Issues - 31/03/2026	49
17.1	Rapida Sintesi su Fasi Learning/Prediction e Capacità di Generalizzazione	49
17.2	Model Selection vs Model Assessment	49
17.3	Validazione su Stima Empirica	49
17.3.1	Hold Out - Ripartizione del Set	49
17.3.2	Pseudoalgoritmo Grid Search - Hold Out	50
17.3.3	Potenziati Errori causati da Cattiva Suddivisione dataset	50
17.3.4	K-Fold Cross Validation	50
17.4	Validazione - Fondamenti Teorici tramite Statistical Learning Theory - SLT	51
17.4.1	Setting Formale della Statistical Learning Theory (SLT)	51
17.4.2	Teoria di Vapnik-Chervonenkis	51
18	— — — — — Lezione 15 - Support Vector Machine (SVM) - 14/04/2026	52
18.1	Definizione SVM e Obiettivi	52
18.1.1	Obiettivo 1: Maximum Margin Classifier	52
18.1.1.1	Rapporto VC-DIM e Massimizzazione Margine	53
18.1.1.2	Hard Margin SVM: Problema di Ottimizzazione Quadratica	53
18.1.1.3	Soft Margin SVM	53
18.1.2	Obiettivo 2: Kernel Trick in Linear Basis Expansion	54
18.1.2.1	Kernel Noti	54
18.1.2.2	Formulazione Finale SVM	54
18.1.3	Obiettivo 3: Evitare Interpretazioni Sbagliate	54
19	— — — — — Lezione 16 - K-NN, Unsupervised Learning e Altri Approcci - 16/04/2026	55
19.1	K-Nearest Neighbors (Supervised Learning)	55
19.1.1	Definizione Formale K-Nearest Neighbors	55
19.1.2	Caratteristiche K-Nearest Neighbors	55
19.2	Approcci di Unsupervised Learning	56
19.2.1	K-Means (Unsupervised Learning)	56
19.2.1.1	Basic K-Means Batch Alg. - (LBG, Generalized Lloyd Alg.)	56
19.2.1.2	Caratteristiche K-Means	57
19.2.2	Dimensionality Reduction via Principal Component Analysis (PCA)	57
19.3	Reinforcement Learning e Semi-Supervised Learning	57
19.4	Neural Networks	57

1 PARTE I: FORMULAZIONE PROBLEMI SU GRAFI E ALGORITMI INFORMATI E NON INFORMATI

2 — — — — — Lezione 1 - Introduzione al Corso - 05/02/2026

Cenni storici ed introduzione. Vedi slide.

3 — — — — — Lezione 2 - Agenti - 05/02/2026

Rif: Slide Lezione_1 - AIMA Cap: 2

3.1 Agenti ed Ambiente

- **Agente:** Un agente razionale **percepisce** il suo **ambiente** tramite **sensori** ed agisce sull'**ambiente** tramite **attuatori**.

Questi possono avere intenzioni e credenze specifiche, questi possono essere sia fisici che virtuali.

- **Percezione e Sequenza Percettiva:** La percezione singola è l'acquisizione di informazioni sull'ambiente da parte dell'agente.
- **Scelta dell'Azione:** Dipende da cosa l'agente ha percepito fino ad ora e cosa ha come conoscenza iniziale.
- **Funzione/Programma Agente:** La funzione agente è una funzione matematica astratta, mentre il programma agente è l'implementazione concreta. Il nostro obiettivo è quello di **implementare il programma agente**.

3.1.1 Razionalità degli Agenti

L'obiettivo è che l'agente interagisca in maniera corretta nell'ambiente secondo **parametri di valutazione oggettiva**.

- **Misura prestazione esterna:** Come vogliamo che il mondo evolva.

Definizione di razionalità basata su 4 fattori:

- **Misura di Prestazione:** Definisce criterio di successo.
- **Conoscenza Pregressa:** dell'ambiente da parte dell'ambiente.
- **Azioni:** che l'agente può eseguire.
- **Sequenza Percettiva:** dell'agente fino all'istante corrente.

Definizione di Agente Razionale: per ogni sequenza di percezioni compie l'azione che massimizza il valore atteso della misura di prestazione considerando le sue percezioni passate e la sua conoscenza pregressa.

Questo **non implica l'onniscienza o l'onnipotenza** dell'agente, ma deve essere in grado di imparare man mano che acquisisce più **percezioni** ed **esperienza**.

3.1.2 Natura degli Ambienti

Gli **ambienti** sono in realtà **definiti come problemi**, cercando di descrivere le caratteristiche del problema, detto anche ambiente operativo, in cui l'**agente deve operare**.

- **Descrizione PEAS:** Descrizione dell'ambiente operativo:
 - **Performance:** Misura di prestazione.
 - **Environment:** Ambiente.
 - **Actuators:** Attuatori.
 - **Sensors:** Sensori.

- **Proprietà degli Ambienti:**

- **Osservabilità:**

- **Ambiente Completamente Osservabile:** I sensori dell'agente permettono di capire lo stato completo dell'ambiente in ogni momento. Se riesce a percepire tutto in maniera completa, non bisogna che l'agente memorizzi tutti gli aspetti acquisiti sull'ambiente.
- **Ambiente Parzialmente Osservabile:** L'agente non può acquisire in ogni istante tutto, quindi bisogna memorizzare le informazioni acquisite.

- **Molteplicità di Agenti:**

- Bisogna distinguere cosa sia e cosa non sia un agente nell'ambiente.
- **Sistemi Multiagente Competitivo:** Ambienti multiagente competitivi potrebbero scontrarsi tra loro per massimizzare la propria misura di prestazione.
- **Sistemi Multiagente Cooperativo:** Gli agenti collaborano per massimizzare la misura di prestazione.

- **Predicibilità:**

- **Deterministico:** Stato successivo completamente determinato dallo stato corrente e dall'azione.
- **Stocastico:** Esistono degli elementi di incertezza con probabilità associata.
- **Non Deterministico:** Esistono vari esiti possibili di una stessa azione, ma non in base ad una probabilità definita.

- **Episodico/Sequenziale:**

- **Episodico:** Esperienza agentica suddivisa in eventi atomici.
- **Sequenziale:** Ogni azione influisce la successiva azione.

- **Ambiente Statico/Dinamico:**

- **Statico:** L'ambiente non cambia mentre l'agente pensa l'azione.
- **Dinamico:** L'ambiente cambia mentre l'agente pensa l'azione.
- **Semid dinamico:** L'ambiente non cambia ma la valutazione della prestazione sì, ad esempio il tempo che scorre in un gioco influisce sulla valutazione della prestazione finale.

- **Stati Discreti/Continui:** Se gli stati definiti sono discreti, o stati continui, come il tempo.

- **Ambiente Noto/Ignoto:** Stato di conoscenza dell'agente circa le leggi fisiche dell'ambiente. Concetto diverso da osservabile, ad esempio l'ambiente di un gioco di carte sarà noto ma parzialmente osservabile, questo perché l'agente conosce le regole del gioco ma non può vedere tutte le carte.

Gli ambienti possono essere simulati per testare l'agente.

3.1.3 Struttura di Agenti

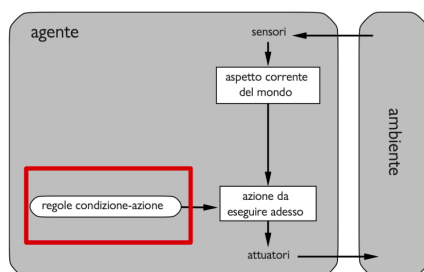
L'**agente** quindi è definito come $\text{Agente} = \text{Architettura} + \text{Programma}$, dove il programma è la funzione $\text{Programma} : \text{Percezione} \rightarrow \text{Azione}$

- **Agente su Tabella:** Si basa sull'accesso all'azione definita in pseudocodice come

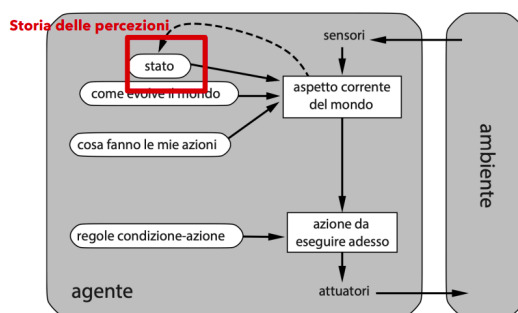
`azione = tabella.get(percezione)`

Ma questo andrebbe in contro ad un esplosione combinatoria, il gioco degli scacchi dovrebbe ad esempio portarsi dietro tutte le possibili combinazioni (sono tante).

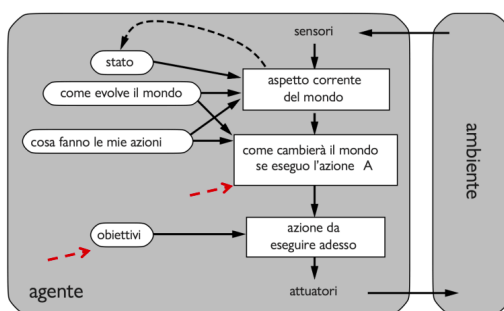
- **Agente Reattivo Semplice:** Si **ignora** la **memoria pregressa**, ma si sceglie l'azione in base all'attuale acquisizione e all'insieme di regole. Questo ci permette di scalare, dato che dovranno essere memorizzate solo le regole, non tutte le possibili sequenze.



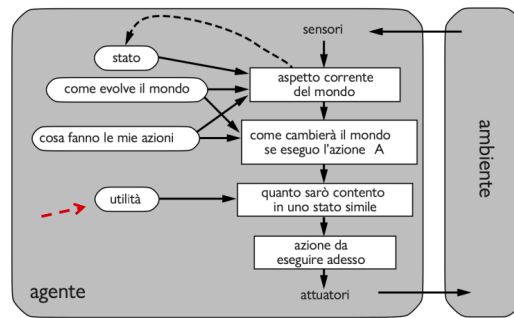
- **Agente Basato su Modello:** L'agente mantiene uno stato interno che serve per tenere traccia della parte del mondo che non può vedere all'istante corrente. Lo stato dipende dalla storia delle percezioni.



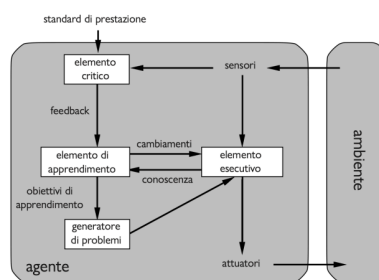
- **Agente Con Obiettivo:** Oltre allo stato, l'agente ha anche un goal, quindi gli obiettivi e la componente di come cambia il mondo eseguendo un'azione.



- **Agente con Funzione di Utilità:** Per dare un peso in termini di utilità agli obiettivi, che potrebbero essere molteplici, si utilizza una **funzione di utilità** con codominio sui reali, per settare ipotetiche priorità tra i vari obiettivi.



- **Agenti che Apprendono:** Qualsiasi tipo di agente visto finora può essere costruito come agente in grado di apprendere. Un agente capace di apprendere può essere diviso in quattro componenti astratti.



3.1.4 Tipi di Rappresentazioni di Ambienti

Esistono varie rappresentazioni di ambiente dell'agente:

- **Atomica:** Ogni stato del mondo è indivisibile.
- **Fattorizzata:** Si suddivide in insiemi di variabili o attributi, ed ognuno di questi può avere un valore.
- **Strutturata:** Suddivide ogni stato in un insieme fissato di variabili o attributi, ognuno dei quali può avere un valore e delle relazioni con altre variabili.

3.2 Risolvere Problemi con la Ricerca

Ricerca: Processo computazionale condotto da un agente risolutore di problemi.

Si **assumono** degli **ambienti semplici**: episodici, a singolo agente, completamente osservabili, deterministici, statici, discreti e noti.

Algoritmi Informati/Non Informati: Se l'agente è in grado rispettivamente di identificare o non identificare la distanza dall'obiettivo.

• Processo di Risoluzione

- **Formulazione obiettivo:** Obiettivo da raggiungere.
- **Formulazione problema:** Elaborazione di un modello astratto descrivendo il mondo reale.
- **Ricerca:** Simulazione in sequenze di azioni, fino a che non trova una sequenza che conduce all'obiettivo.
- **Esecuzione:** Eseguire le azioni specificate nella soluzione.

3.2.1 Definizione Problemi di Ricerca e Soluzioni

Possiamo definire un problema di ricerca tramite 6 elementi:

- **Spazio degli stati:** Insieme di possibili stati in cui si può trovare l'ambiente.
- **Stato iniziale:** Lo stato in cui si trova l'agente inizialmente.
- **Stato obiettivo:** Può essere uno solo o più di uno, quindi qualcosa del genere $\text{goalTest}(\text{state}) \in \{\text{True}, \text{False}\}$
- **Azioni:** Insieme finito di azioni che possono essere eseguite in uno stato. Ad esempio $\text{Azioni}(\text{Stato}) = \{\text{Azione}_1, \dots, \text{Azione}_n\}$
- **Modello di transizione:** Descrizione dell'azione, del tipo $\text{Risultato}(\text{Stato}, \text{Azione}) = \text{Stato}'$
- **Funzione Costo:** Definisce il costo di eseguire una specifica azione partendo da uno specifico stato: $\text{CostoAzione}(\text{Stato}, \text{Azione}, \text{Stato}')$

Quindi un **cammino** rappresenta una **sequenza di azioni**, una **soluzione** è un **cammino** che parte dallo **stato iniziale** ed arriva ad uno **stato obiettivo**. Tra tutte le soluzioni ne esiste una **ottimale**, ossia di **costo minimo**.

4 — — — — Lezione 3 - Algoritmi di Ricerca e Strategie

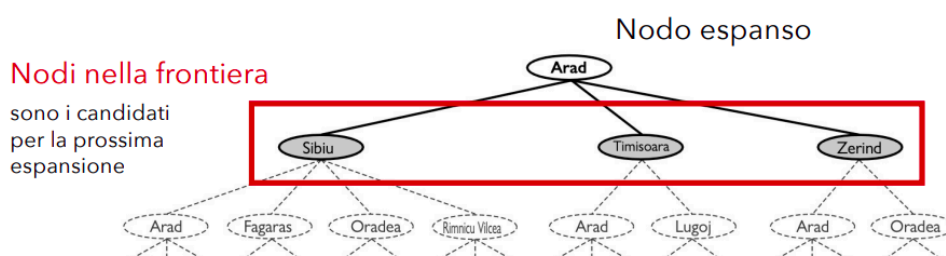
Ricerca non Informata - 10/02/2026

Rif: Slide Lezione_2 - AIMA Cap: 3, Sez: 3.3, 3.4

Ripartendo dalle definizioni della lezione successiva, ricordiamo che spazio stati = albero di ricerca.

4.1 Algoritmo di Ricerca

- **Espansione di un Nodo:** Considerando le azioni disponibili nello stato corrispondente, si genera un nuovo nodo figlio nell'albero di ricerca in corrispondenza di ciascuna delle azioni possibili dato lo stato corrente.
 - **Frontiera:** insieme dei nodi generati ma non ancora espansi. Frontiera perchè rappresenta un confine tra **regione interna** espansa e **regione esterna** di stati non ancora raggiunti.
 - **Stati Raggiunti:** stati del grafo per cui esiste un corrispondente generato nell'albero di ricerca.



Quindi una ricerca va avanti espandendo ogni volta un nodo della frontiera. L'**algoritmo di ricerca** specifico quindi si occuperà di **scegliere quale** tra i **nodi di frontiera** bisogna **sviluppare**.

4.1.1 Strategia Best-First

Si sceglie il nodo sulla frontiera in cui $f(n)$ ha valore minimo:

- **Nuovo nodo non raggiunto:** sicuramente viene aggiunto in frontiera.
- **Nuovo nodo già raggiunto:** viene aggiunto in frontiera solo se migliora il costo di valutazione.

Questo algoritmo iterativo conclude o con fallimento oppure con un cammino che porta ad un obiettivo.

Questa è più una strategia, non un algoritmo reale, perchè non abbiamo definito cosa sia la specifica $f(n)$, e si sta seguendo un approccio greedy.

```
function RICERCA-BEST-FIRST(problema, f) returns un nodo soluzione o fallimento
  nodo ← NODO(STATO=problema.STATOINIZIALE)
  frontiera ← una coda con priorità ordinata in base a f, con nodo come elemento iniziale
  raggiunti ← una tabella di lookup, con un elemento con chiave problema.STATOINIZIALE e valore nodo
  while not VUOTA?(frontiera) do
    nodo ← POP(frontiera)
    if problema.È-OBIETTIVO(nodo.STATO) then return nodo
    for each figlio in ESPANDI(problema, nodo) do
      s ← figlio.STATO
      if s non è in raggiunti or figlio.COSTO-CAMMINO < raggiunti[s].COSTO-CAMMINO then
        raggiunti[s] ← figlio
        aggiungi figlio a frontiera
  return fallimento
```

4.1.2 Strutture Dati per la Ricerca

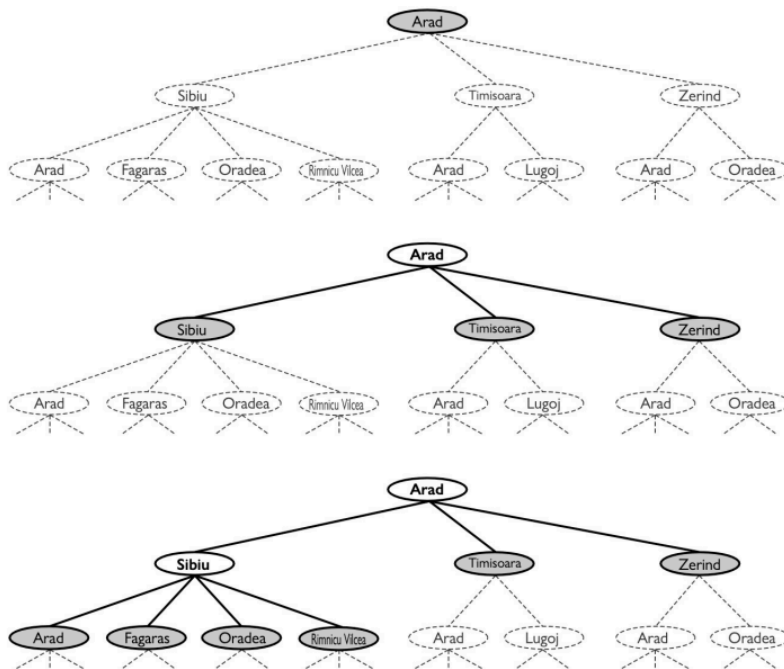
- **Nodo:** Immaginando una struttura ad oggetto:
 - *n.stato*: Lo stato a cui corrisponde il nodo.
 - *n.padre*: Il nodo dell'albero di ricerca che ha generato il nodo corrente.
 - *n.azione*: L'azione applicata allo stato del padre per generare il nodo corrente.
 - *n.costoCammino*: Costo totale del cammino dallo stato iniziale a questo nodo.
- **Frontiera (Coda):**
 - *frontiera.checkVuota*: ritorna true/false in base a se sia o meno vuota.
 - *frontiera.pop*: rimuove elemento in cima.
 - *frontiera.top*: riferisce l'elemento in cima.
 - *frontiera.add*: inserisce nodo secondo l'implementazione della coda.

Esistono quindi implementazioni diverse di coda in base a che logica forniamo a questi metodi quindi ad esempio **coda con priorità** (su cui si basa la best-first), **coda FIFO** (su cui si basa la ricerca in ampiezza) o **coda LIFO** (su cui si basa la ricerca in profondità).

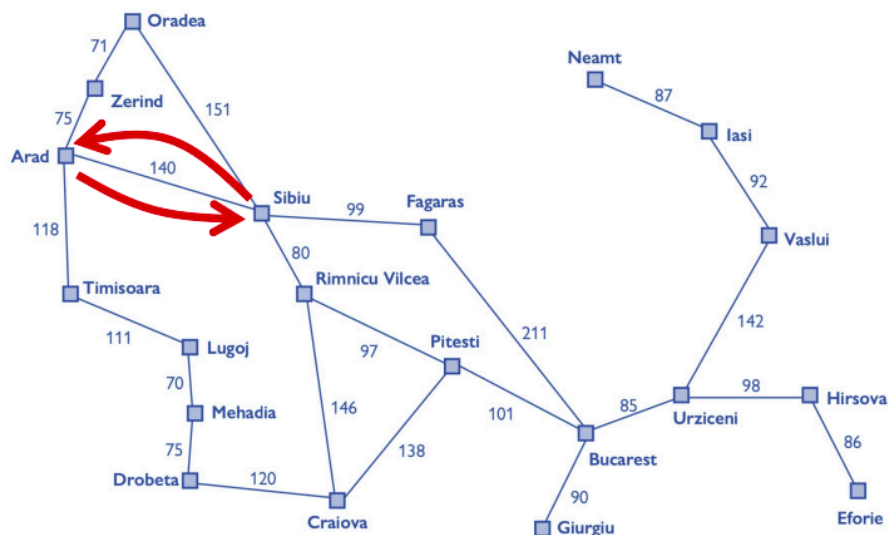
- **Stati Raggiunti (Tabella Lookup):** Ogni stato è una chiave, ogni valore è il nodo per tale stato.

4.1.3 Concetti di Stato Ripetuto, Ciclo, Cammini Ridondanti

- **Stato Ripetuto:** Stato che corrisponde a più nodi nello stesso cammino.



- **Cammino Ciclico:** Anche dato uno spazio degli stati finito, l'albero di ricerca generato è infinito a causa dei cicli.



- **Cammino Ridondante:** Una via peggiore per raggiungere qualcosa di già raggiunto.
 - **Ricerca su Grafo:** Viene controllata la presenza di cammini ridondanti.
 - **Ricerca ad Albero:** Non viene controllata la presenza di cammini ridondanti.

Esistono specifici **approcci** per controllare **cammini ridondanti**:

- **Ricordare** tutti gli stati **precedentemente raggiunti**.

- **Non preoccuparsi dei cammini ridondanti.**
- Controllare presenza di **cammini ciclici** (sottocaso dei cammini ridondanti), ma non quindi il superinsieme di quelli ridondanti. Quindi questo è un po' una via intermedia tra i due approcci sopra.
 - Si potrebbe ad esempio controllare tramite la catena di puntatori mantenuta da tutti i nodi.

4.2 Misure di Prestazione: Valutazione di Bontà Algoritmo

Quattro parametri per la valutazione di algoritmi:

- **Completezza:** Garantisce di riportare una soluzione, se esiste, e di riportare correttamente il fallimento, se esiste. Deve quindi procedere in maniera sistematica.
- **Ottimalità rispetto al costo:** L'algoritmo trova la soluzione con il costo di cammino minimo.
- **Complessità temporale:** Quanto tempo impiega l'algoritmo a trovare una soluzione.
- **Complessità spaziale:** Quanta memoria utilizza l'algoritmo per effettuare la ricerca.

4.2.1 Misurazione della Complessità

Se il grafo viene espresso in maniera esplicita allora la complessità è formulata in termini di $|V| + |E|$, ma in pratica si misura in termini di:

- **Profondità dell'albero generato (d):** quindi dimensione in numero di azioni di una soluzione ottima.
- **Massima profondità (m):** Numero massimo di azioni in un qualsiasi cammino ($d \leq m$).
- **Branching Factor (b):** Numero di successori di un nodo.

4.3 Algoritmo Ricerca in Ampiezza

Appropriata quando tutte le azioni hanno lo stesso costo.

- **Implementazione 1:** Tramite istanziamento della BestFirst, settando $f(n) = \text{depth}(n)$.
- **Implementazione 2:** Tramite utilizzo di una coda FIFO per la frontiera, dove i nodi inseriti nella frontiera verranno espansi in ordine di inserimento.
 - Questi nodi raggiunti quindi non verranno mantenuti in una lookup table, ma tramite la coda stessa.

```

function RICERCA-IN-AMPIEZZA(problema) returns un nodo soluzione o fallimento
  nodo  $\leftarrow$  NODO(problema.STATOINIZIALE)
  if problema.È-OBIETTIVO(nodo.STATO) then return nodo
  frontiera  $\leftarrow$  una coda FIFO, con nodo come elemento iniziale
  raggiunti  $\leftarrow$  {problema.STATOINIZIALE}
  while not VUOTA?(frontiera) do
    nodo  $\leftarrow$  POP(frontiera)
    for each figlio in ESPANDI(problema, nodo) do
      s  $\leftarrow$  figlio.STATO
      if problema.È-OBIETTIVO(s) then return figlio
      if s non è in raggiunti then
        aggiungi s a raggiunti
        aggiungi figlio a frontiera
  return fallimento

```

4.3.1 Analisi Misure Prestazione Ricerca in Ampiezza

- **Completa:** ricerca sistematica
- **Ottima rispetto al costo:** Se ogni azione costa 1, allora lo è, altrimenti non è ottima.
- **Costo Temporale e Spaziale:** Scala esponenzialmente, data la b branching factor allora $\sum_{i=0}^d b^i = O(b^d)$. Quindi il **costo in memoria è esponenziale**. Quindi può essere utilizzato solo su istanze piccole.

4.4 Algoritmo di Dijkstra (a Costo Uniforme)

Va bene se le azioni hanno tra loro costi diversi.

Si può **implementare tramite bestFirst** con $f(a)$ settata come costo del cammino dalla radice fino al nodo corrente.

```
function RICERCA-COSTO-UNIFORME(problema) returns un nodo soluzione o fallimento  
return RICERCA-BEST-FIRST(problema, COSTO-CAMMINO)
```

4.4.1 Analisi Misure Prestazione Ricerca a Costo Uniforme

- **Completa:** è sistematica, e non entra mai in ciclo infinito sull'assunzione che ogni costo sia sempre positivo.
- **Ottima rispetto al costo:** La prima soluzione che trova ha un costo basso almeno come quello di ogni altro nodo sulla frontiera.
- **Complessità:**
 - C^* : costo della soluzione ottima.
 - ε : limite inferiore imposto al costo di ogni azione
 - $\lfloor \frac{C^*}{\varepsilon} \rfloor$: numero maggiore possibile di azioni che servono per raggiungere il costo della soluzione ottima. Ci permette di supporre una profondità massima.
 - **Costo in tempo e spazio:** $O\{1 + \lfloor \frac{C^*}{\varepsilon} \rfloor\}$, scala quindi esponenzialmente.

4.5 Algoritmo Ricerca in Profondità

Si sviluppa il nodo più in profondità.

- **Implementazione 1:** Tramite istanziamento della BestFirst, settando $f(n) = -\text{depth}(n)$.
- **Implementazione 2:** Tramite utilizzo di una coda LIFO, quindi tramite ricerca ad albero senza tabella dei raggiunti.

4.5.1 Analisi Misure Prestazione Ricerca in Profondità

- **Completa:**
 - Per spazi degli stati finiti e che sono alberi, è completa ed efficiente.
 - Per spazi degli stati aciclici resta sistematica
 - Per spazi con cicli può bloccarsi in cicli infiniti
 - Per spazi **stati infiniti non è sistematica**, quindi **incompleta**.
- **Costo in Tempo e Spazio:** Devo tenere in memoria solo la frontiera, quindi **memoria** $O(bm)$, mentre **tempo** $O(b^m)$.
 - In caso di **backtracking**, espandendo un nodo viene generato solo un successore e si ricorda quale deve essere generato in seguito, passando in **memoria** a $O(m)$.

4.6 Algoritmo Ricerca in Profondità Limitata

Si basa su un limite settato l di profondità, per evitare che quest'ultimo si perda in un cammino infinito. Il valore l può essere scelto in base a:

- In base alla conoscenza del problema
- Diametro dello spazio degli stati

```
function RICERCA-PROFONDITÀ-LIMITATA(problema,  $l$ ) returns un nodo soluzione o fallimento o soglia
  frontiera  $\leftarrow$  una coda LIFO (stack) con NODO(problema.STATOINIZIALE) come elemento iniziale
  risultato  $\leftarrow$  fallimento
  while not VUOTA?(frontiera) do
    nodo  $\leftarrow$  POP(frontiera)
    if problema.È-OBIETTIVO(nodo.STATO) then return nodo
    if PROFONDITÀ(nodo)  $> l$  then
      risultato  $\leftarrow$  soglia
    else if not È-CICLO(nodo) do
      for each figlio in ESPANDI(problema, nodo) do
        aggiungi figlio a frontiera
  return risultato
```

Soglia: valore di output che segnala che potrebbe esistere una soluzione a un livello di profondità maggiore di l

4.6.1 Analisi Misure Prestazione Ricerca in Profondità Limitata

- **Completezza:** Dipende dalla scelta di l .
- **Non Ottimale.**
- **Complessità Temporale:** $O(b^l)$
- **Complessità Spaziale:** $O(bl)$

4.7 Algoritmo Ricerca ad Approfondimento Iterativo

Si cerca iterativamente il valore di l invocando l'algoritmo definito prima, quindi a ricerca limitata.

```
function RICERCA-PROFONDITÀ-LIMITATA(problema,  $l$ ) returns un nodo soluzione o fallimento o soglia
  frontiera  $\leftarrow$  una coda LIFO (stack) con NODO(problema.STATOINIZIALE) come elemento iniziale
  risultato  $\leftarrow$  fallimento
  while not VUOTA?(frontiera) do
    nodo  $\leftarrow$  POP(frontiera)
    if problema.È-OBIETTIVO(nodo.STATO) then return nodo
    if PROFONDITÀ(nodo)  $> l$  then
      risultato  $\leftarrow$  soglia
    else if not È-CICLO(nodo) do
      for each figlio in ESPANDI(problema, nodo) do
        aggiungi figlio a frontiera
  return risultato
```

Soglia: valore di output che segnala che potrebbe esistere una soluzione a un livello di profondità maggiore di l

Questo potrebbe sembrare uno spreco di iterazioni, ma aumentando profondità sappiamo che l'algoritmo in profondità scala esponenzialmente, quindi in relazione a questo le iterazioni in più non sono significative.

4.7.1 Analisi Misure Prestazione Ricerca ad Approfondimento Iterativo

- **Completezza:** Completa su spazi di stati aciclici finiti, o su qualsiasi spazio degli stati finito se controlliamo la presenza di cicli risalendo l'intero cammino.
- **Ottima per problemi** in cui tutte le **azioni** hanno lo **stesso costo**.
- **Complessità Temporale:** $O(b^d)$ se esiste una soluzione, $O(b^m)$ se non esiste una soluzione.
- **Complessità Spaziale:** $O(bd)$ se esiste una soluzione, $O(bm)$ se non esiste.

4.8 Direzione della Ricerca

Esiste ricerca in avanti statoIniziale $\rightarrow$ obiettivo o all'indietro obiettivo $\rightarrow$ statoIniziale.

- **All'indietro** si può applicare solo se l'obiettivo è ben definito.
- **In avanti** quando ci sono più obiettivi

4.8.1 Ricerca Bidirezionale

Stesso algoritmo ma due problemi di ricerca, dove nel **forward** si **setta normalmente stato iniziale e obiettivo**, mentre nel **backward** si **invertano** questi due.

Si procede quindi in entrambe le direzioni fino ad incontrarsi.

Bisogna quindi tenere traccia di **due frontiere** e **due tabelle di raggiunti**.

Non si procede però espandendo due nodi contemporaneamente, si sceglie infatti un solo nodo in una delle due frontiere.

4.8.2 Analisi Misure Prestazione Ricerca Bidirezionale

- **Complessità Tempo:** $O\left(\frac{b^d}{2}\right)$.
- **Complessità Spazio:** $O\left(\frac{b^d}{2}\right)$.

4.9 Confronto Complessivo Misure Prestazione degli Algoritmi

Confronto

b – branching factor m – massima profondità albero di ricerca
 d – profondità della soluzione ottima l – limite alla profondità

criterio	in ampiezza	a costo uniforme	in profondità	a profondità limitata	ad approfondimento iterativo	bidirezionale (se applicabile)
completa?	SI ¹	SI ^{1,2}	No	No	SI ¹	SI ^{1,4}
ottima rispetto al costo?	SI ³	SI	No	No	SI ³	SI ^{3,4}
tempo	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
spazio	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b^l)$	$O(bd)$	$O(b^{d/2})$

- ¹ – se b è finito, e lo spazio degli stati ha una soluzione o è finito
- ² – completa se tutti i costi di azione sono $\geq \epsilon > 0$
- ³ – ottima rispetto al costo se i costi delle azioni sono tutti identici
- ⁴ – se in entrambe le direzioni si usa una ricerca in ampiezza o una ricerca a costo uniforme

Quindi tutti questi sono detti di ricerca non informata perchè si basano solo sulla definizione del problema.

5 — — — — Lezione 4 - Strategie di Ricerca Informata o Euristiche - 12/02/2026

Rif: Slide Lezione_3 - AIMA Cap: 3, Sez: 3.5, 3.6

La ricerca informata si basa sulla conoscenza specifica di dove si potrebbe trovare l'obiettivo per accelerare la ricerca nello spazio degli stati.

Funzione Euristiche: si definisce una funzione $h(n)$, costo stimato del cammino.

5.1 Ricerca Best-First Greedy

Espandere prima il nodo con il valore più basso di $h(n)$, quindi $f(n) = h(n)$, con $f(n)$ il costo della Best-First.

- Esempio con distanza in linea d'aria, oltre alle informazioni sul problema descritto, abbiamo anche una conoscenza sulla funzione h .

Si basa solo sull'euristica.

5.1.1 Analisi di Best First Greedy:

- Completa in spazi di stati finiti, ma non in quelli infiniti.
- Non ottima rispetto al costo
- Complessità spaziale e temporale $O(|V|)$

5.2 Ricerca A*

Ricerca best-first che usa la seguente funzione di valutazione:

$$f(n) = g(n) + h(n)$$

dove

- $f(n)$: costo stimato del cammino migliore che continua da n fino ad un nodo

obiettivo

- $g(n)$: costo del cammino dal nodo iniziale ad n .
- $h(n)$: costo stimato del cammino più breve da n ad uno stato obiettivo.

Quindi il concetto fondamentale è tener conto sia del costo fino ad ora sia il costo dell'euristica in $f(n)$.

5.2.1 Analisi A*

- Completa
- Ottima rispetto al costo se l'euristica $h(n)$ è **ammissibile**.

5.2.2 Ammissibilità di Euristica

Un **euristica** è **ammissibile** se **non sovrastima** mai il **costo** per raggiungere un obiettivo, quindi se $h(n) \leq h^*(n)$, dove $h^*(n)$ è il costo minimo di un cammino da n fino ad un goal.

5.2.3 Dimostrazione: Se $h()$ ammissibile allora A* è ottimo

Si imposta per assurdo:

A^* restituisce un cammino di costo C , ma esiste un $C^* < C$

Allora certamente esiste un nodo n sul cammino ottimo che non è stato espanso.

Denotiamo con

- $g^*(n)$ costo del cammino ottimo dall'inizio fino ad n
- $h^*(n)$ costo del cammino ottimo da n fino ad un obiettivo

Quindi:

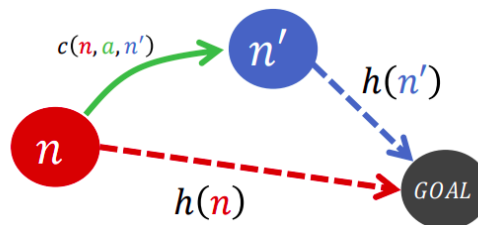
- $f(n) > C^*$, altrimenti n sarebbe stato espanso.
- $f(n) = g(n) + h(n)$, per definizione.
- $f(n) = g^*(n) + h(n)$, siccome n si trova su un cammino ottimo.
- $f(n) \leq g^*(n) + h^*(n)$, siccome $h(n)$ è ammissibile allora $h(n) \leq h^*(n)$.
- $f(n) \leq C^*$, dato che per definizione (vedi slide)

quindi per 1. e 5. abbiamo raggiunto l'assurdo.

5.2.4 Euristiche Consistenti

Un'euristica è **consistente** se, per ogni nodo n , e per ogni successore n' di n generato da un'azione a risulta essere:

$$h(n) \leq c(n, a, n') + h(n')$$



Consistente implica **ammissibile**, più forte come condizione.

Se h è un'euristica consistente allora $f = g + h$ non decresce mai lungo i cammini.

5.2.5 Euristiche Inconsistenti

Possiamo trovarci con **più cammini** che raggiungono lo **stesso stato**.

Se ogni cammino nuovo ha costo inferiore al precedente finiremo con l'avere più nodi corrispondenti a quello stato sulla frontiera.

Inconsistente non implica non ammissibile.

5.2.6 Algoritmo A* con Euristiche Consistenti, Inconsistenti, Inammissibili

- **A* con Euristiche Consistenti:**
 - **Ottimo rispetto al costo.**
 - La **prima volta** che **raggiungiamo uno stato** sarà su un **cammino ottimo**.
- **A* con Euristiche Inconsistenti:**
 - Possiamo trovarci con **più cammini** che raggiungono lo **stesso stato**.
 - Se ogni cammino nuovo ha costo inferiore al precedente finiremo con l'avere più nodi corrispondenti a quello stato sulla frontiera, portando ad un **aggravio di costo spaziale e temporale**.
- **A* con Euristiche Inammissibili:**
 - Può essere ottimo rispetto al costo in due casi
 - Esiste un **cammino ottimo** rispetto al costo lungo cui $h(n)$ è **ammissibile per tutti i nodi sul cammino**.
 - Se la soluzione ottima ha costo C^* e la seconda soluzione migliore ha costo C_2 , e se $h(n)$ sovrastima i costi, ma al massimo di una quantità $k \leq C_2 - C^*$

5.2.7 Confini della Ricerca A e Caratteristiche A

- Si basa sullo sviluppo di bande concentriche basate sulla f .
- **Caratteristiche di A***: Assumendo che C^* sia costo del cammino della soluzione ottima:
 - **A* espande tutti i nodi** che possono essere raggiunti dallo stato iniziale su **un cammino in cui per ogni nodo** si ha $f(n) < C^*$.
 - **A* può espandere** alcuni dei nodi sul confine obiettivo, quindi dove $f(n) = C^*$ prima di selezionare il nodo obiettivo.
 - **A* non espande** alcun nodo con $f(n) > C^*$

5.2.8 A* come Algoritmo Ottimamente Efficiente

Fornendo ad A* un **euristica consistente** è **ottimamente efficiente**, quindi qualsiasi altro algoritmo che estente cammini di ricerca a partire dallo stato iniziale e sua la stessa euristica deve espandere tutti i nodi che sono certamente espansi da A*.

5.2.9 Ricerca Soddisfacente

Il problema di A* è che **espande un grande numero di nodi**, di conseguenza preferiamo a volte accettare **soluzioni subottime** ma **sufficientemente buone**, solo per **diminuire in numero di nodi esplorati**.

5.2.10 Ricerca A* Pesata

Viene definita un $W > 1$ fattore moltiplicativo per dare maggiore peso al valore dell'euristica, quindi $f(n) = g(n) + Wh(n)$

In questo modo possiamo esplorare meno stati con un trade-off sul costo, quindi assumendo che la soluzione ottima abbia costo C^* , allora troverà una soluzione di costo compreso tra C^* e $W \times C^*$.

In realtà la **ricerca A* pesata** può essere vista come una **generalizzazione delle altre**.

Ricerca A*:	$g(n) + h(n)$	$(W = 1)$
Ricerca a costo uniforme:	$g(n)$	$(W = 0)$
Ricerca best first greedy:	$h(n)$	$(W = \infty)$
Ricerca A* pesata:	$g(n) + W \times h(n)$	$(1 < W < \infty)$

5.2.11 Varianti A* per Limitazioni

- **Algoritmi di Ricerca Subottima**:
 - **Ricerca a subottimalità limitata**: Cerchiamo una soluzione con granzia che si trovi entro un fattore costante W del costo ottimo, come A* pesata.
 - **Ricerca a costo limitato**: cerchiamo una coluzione il cui costo sia inferiore ad una costante C .
- **Ricerca a costo illimitato**: Accettiamo una soluzione di qualsiasi costo, purchè riusciamo a trovarla rapidamente.
- **Ricerca con Memoria Limitata**: Si tenta tramite due ottimizzazioni di migliorare l'utilizzo di memoria dell'algoritmo A*.
 - Gli stati sono **presenti** solo in una delle due posizioni, quindi **frontiera** o **raggiunti**.

- Rimuovere gli stati dai raggiunti quando si può dimostrare che non servono più.
- **Ricerca Beam**: Limitare la **dimensione della frontiera**, mantenendo solo i k nodi con i migliori costi f . Questo definisce **incompletezza** e **subottimalità**, esplorando solo un settore dei confini concentrici.
- **Ricerca A* con Approfondimento Iterativo - (IDA*)**: Ricerca ad approfondimento iterativo combinata con A*, quindi non tiene in memoria tutti gli stati raggiunti. La soglia è settata sul valore $f = g + h$.
 - Si effettua quindi, per ogni iterazione, una ricerca esaustiva in profondità fino ad un costo f_l , quindi quando viene trovato un nodo n con costo $f(n) > f_l$, termina iterazione e ricomincia la successiva con $f_l = f(n)$.
- **Ricerca Best First Ricorsiva - (RBFS)**: Ricerca in BestFirst, con interruzione quando f supera un valore f_{limite} . Se viene superato questo limite, la ricorsione torna al cammino alternativo.
- **Memory Bounded A* - (SMA*)**: Si basa sull'utilizzo di **tutta la memoria disponibile** fino al suo esaurimento, poi **dimentica** il nodo peggiore memorizzando nel nodo del padre il valore del nodo dimenticato.

5.3 Funzioni Euristiche

- Nel caso del rompicapo degli 8 tasselli possiamo immaginare 2 esempi di funzione h euristica:
 - h_1 numero di tasselli fuori posto.
 - h_2 somma delle distanze di tutti i tasselli dalla loro posizione corrente alla configurazione obiettivo.

Ciascuna di queste funzioni euristiche ha delle caratteristiche:

- **Fattore di ramificazione effettivo**: Si basa sugli elementi:
 - N numero totale di nodi generato da A*.
 - d profondità dell'albero generato da A*.
 - b^* fattore di ramificazione che un albero uniforme di profondità d dovrebbe avere per contenere $N + 1$ nodi, quindi:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

Una buona funzione euristica dovrebbe avere un valore di b^* vicino ad 1.

5.3.1 Dominazione di Euristiche

Date due euristiche h_1, h_2 si dice che h_1 **domina** h_2 se per ogni nodo n vale che

$$h_1(n) \geq h_2(n)$$

5.4 Costruzione di Euristiche

5.4.1 Euristica da Problema Rilassato

Costruzione del problema originale con meno vincoli sulle azioni che si possono compiere. Potremmo ad esempio immaginare di aggiungere archi allo spazio degli stati.

Quindi il costo di una soluzione ottima per un problema rilassato è un euristica ammissibile per il problema originale.

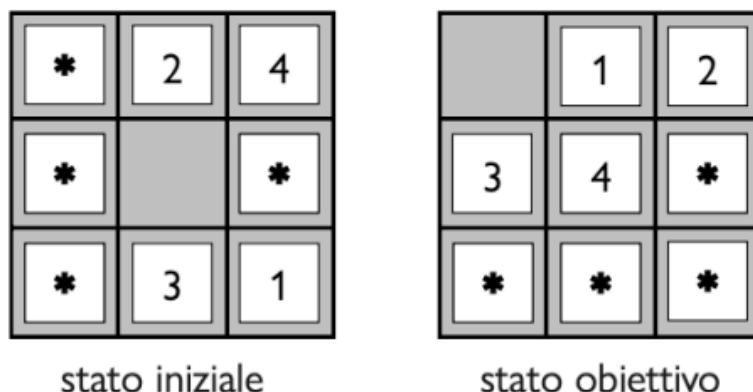
5.4.2 Massimizzazione di Insieme di Euristiche Ammissibili

Dato un insieme di euristiche ammissibili $h_1, \dots, h_m$ costruiamo $h(n) = \max\{h_1(n), \dots, h_m(n)\}$.

Se tutte le funzioni $\{h_1(n), \dots, h_m(n)\}$ sono ammissibili allora anche h lo è, e **domina** tutte le altre.

5.4.3 Euristiche da Sottoproblema

Un altro metodo per costruire euristiche è quello di tenere in considerazione solo una parte del problema originale.



Il **costo della soluzione** ottimale di un **sottoproblema** è una **sottostima del costo del problema originale**, quindi è un **euristica ammissibile**.

5.4.4 Database di Pattern Soluzioni

Possiamo **memorizzare** i costi esatti delle soluzioni di ogni possibile istanza di un sottoproblema, definendo quindi un **euristica ammissibile** h_{db} per ogni stato **incontrato durante la ricerca**, estraendo dal database la configurazione corrispondente del sottoproblema.

Non possiamo però a priori sommare i valori di ciascuna soluzione di istanza, dato che le soluzioni ai sottoproblemi possono interferire.

- **Landmarks:** Tramite il **pre-calcolo** di alcuni **costi di cammino ottimo** ed il **salvataggio rispetto a punti di riferimento rilevanti** è possibile ottimizzare il calcolo effettivo a tempo di esecuzione dell'algoritmo.
- **Apprendere dall'esperienza:** Si basa sull'apprensione di istanze del problema già risolte, basandosi sull'utilizzo di features dello stato fornite per predire il valore dell'euristica.
 - Nell'esempio dei tasselli potremmo indicare con x_1 in **numero di tasselli fuori posto**, x_2 **numero di tasselli adiacenti che sono adiacenti anche nella soluzione** e c_1, c_2 **appresi tramite algoritmo di apprendimento** formulando quindi l'euristica come:

$$h(n) = c_1 x_1(n) + c_2 x_2(n)$$

6 — — — — — Lezione 5 - Ricerca Locale e Cenni di Ricerca Online - 17/02/2026

Rif: Slide Lezione_4 - AIMA Cap: 4, Sez: 4.1, 4.2, 4.3

Fino ad ora abbiamo assunto **forti condizioni**:

- Ambienti **deterministici** e **completamente osservabili**.
- Agenti in grado di produrre offline un piano che può essere eseguito senza imprevisti.

Potremmo quindi tenere conto di ambienti un po' più realistici, quindi:

- L'agente ha bisogno di un **piano condizionale** (basato su OR, AND, IF...), in un mondo non deterministico, quindi rilassamento del determinismo.
- Ricerca di stati buoni in spazi discreti o continui.
- **Rilassamento** della condizione per cui tutti gli **stati** sono **completamente osservabili**, gli agenti avranno visibilità parziale degli stati.
- **Rilassamento del vincolo sull'ambiente noto**, guidando l'agente attraverso uno spazio sconosciuto, mediante ricerca online.

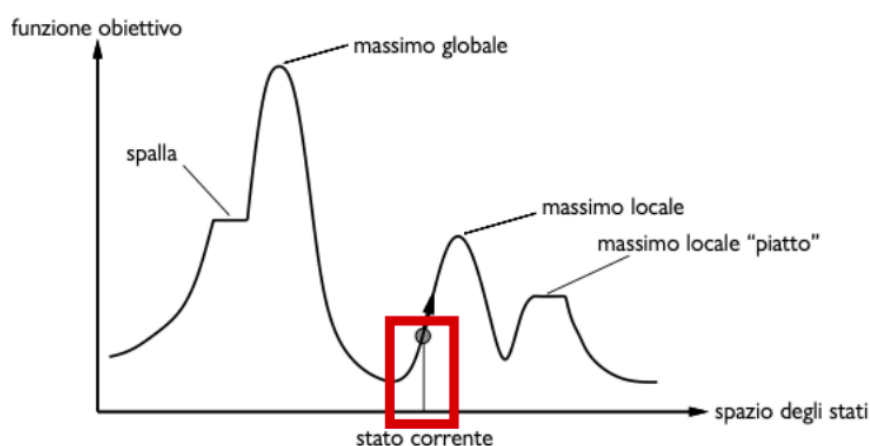
6.1 Ricerca Locale

Gli algoritmi di ricerca locale operano a partire da uno stato iniziale e procedono verso gli stati adiacenti, **senza tenere traccia dei cammini** e **senza tenere traccia degli stati già raggiunti**.

- Quindi **per definizione** questi **algoritmi non** sono **sistematici**, quindi potrebbero non esplorare un sottospazio degli stati dove risiede una soluzione.
- Usano poca memoria e riescono a trovare stati goal (soluzione) in spazi degli stati molto grandi.

6.1.1 Problemi di Ottimizzazione (Hill Climbing, Simulated Annealing, Local Beam Search)

Lo scopo è quello di **trovare lo stato migliore** secondo una **funzione obiettivo**.



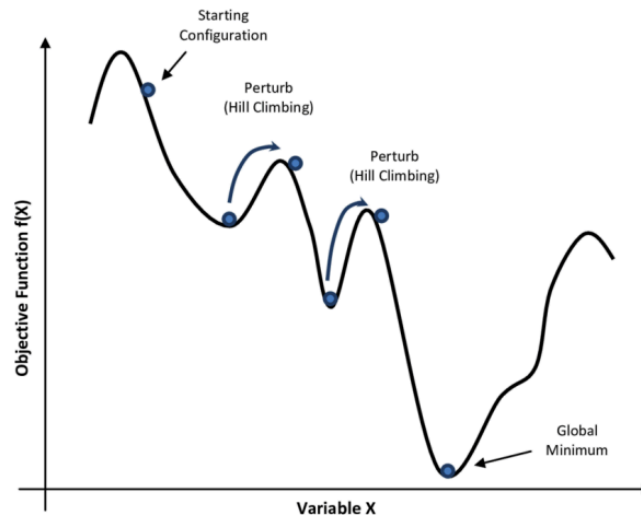
Esistono vari tipi di ricerca dell'ottimo:

- **Ricerca Hill Climbing:**

- Ricerca **locale greedy**, si tiene conto solo dello stato corrente.
- Si cerca di andare nella posizione che porta ad un incremento di altezza.
- Si **fermerà** quindi al **primo massimo locale** che trova.
- Per un minimo locale si utilizza lo stesso algoritmo, negando la funzione obiettivo.

```
function HILL-CLIMBING(problema) returns uno stato che è un massimo locale
  corrente ← problema.STATOINIZIALE
  while true do
    vicino ← lo stato successore di corrente di valore più alto
    if VALORE(vicino) ≤ VALORE(corrente) then return corrente
    corrente ← vicino
```

- **Soffre vari pattern comuni** dato il fatto che cerca degli ottimi locali, quindi ad esempio **creste, plateau...**
- Nel problema delle 8 regine ad esempio l'algoritmo hill climbing **si blocca** nel 86% delle volte e **trova la soluzione** il 14% delle volte, ma è **molto veloce**.
- **Miglioramenti Hill Climbing:**
 - **Mossa Laterale:** Si **permette l'esplorazione dei plateau**, permettendo quindi un **numero limitato di passi** anche se il valore della **funzione obiettivo resta invariato**. Questo nel caso del problema delle 8 regine porta ad un successo il 94% delle volte.
 - **Hill Climbing Stocastico:** Si sceglie a caso tra tutti gli stati che portano ad una crescita della funzione obiettivo. Converge più lentamente dell'originale ma può trovare soluzioni migliori.
 - **Hill Climbing Con Prima Scelta:** Generare a caso i successori dello stato attuale, fino ad ottenerne uno preferibile a quello corrente.
 - **Hill Climbing Con Riavvio Casuale:** Serie di ricerche hill climbing partendo da stati iniziali generati casualmente, fino a che non viene raggiunto un obiettivo.
 - **Tradeoff** quindi di **efficienza per completezza**.
 - Data p probabilità di successo per l'algoritmo Hill Climbing, allora il numero atteso di riavvii sarà $\frac{1}{p}$.
- **Successo ed insuccesso dipendono** quindi fortemente dalla **forma dello spazio degli stati**, ma in un contesto reale è molto probabile che ci sia uno spazio degli stati eterogeneo.
- **Simulated Annealing:**
 - Cerca di **combinare Hill Climbing ed Esplorazione Casuale**, quindi rispettivamente non scendere mai e scelta casuale.
 - Si cerca di **perturbare** il risultato di **Hill Climbing** per cercare di **uscire da punti di minimo locale** (si ragiona in termini di minimizzazione).



- Il funzionamento si basa sull'**accettare o meno una scelta casuale**:
 - Se la mossa è migliorativa allora viene accettata.
 - Altrimenti viene accettata secondo una probabilità $\text{prob} < 1$.
 - La **probabilità** deve **decrescere esponenzialmente** rispetto a quanto **peggiora la valutazione** ΔE , ed oltre a questo **decresce** con la **temperatura** T , che scende costantemente, quindi

$$\text{prob} = e^{-\frac{\Delta E}{T}}$$

- Il valore della temperatura T diminuisce rispetto ad una velocità di raffreddamento. Se questo valore decresce abbastanza lentamente allora l'algoritmo troverà un massimo globale con probabilità che tende ad 1.

```

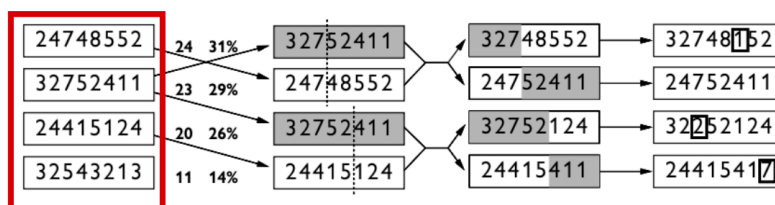
function simulated-annealing(problema, velocità_raffreddamento) returns uno stato soluzione
  corrente ← problema.StatoIniziale
  for t ← 1 to ∞ do
    T ← velocità_raffreddamento[t]
    if T = 0 then return corrente
    successivo ← un successore scelto a caso di corrente
    ΔE ← Valore(successivo) - Valore(corrente)
    if ΔE < 0 then corrente ← successivo
    else corrente ← successivo solo con probabilità exp(-ΔE/T)

```

- **Local Beam Search**:
 - Si tiene traccia di k stati, quindi un intervallo attorno allo stato corrente invece che uno solo.
 - Questo permette l'abbandono preventivo a situazioni infruttuose.
 - Esiste anche una **versione stocastica**, che sceglie i **successori** con **probabilità** proporzionale al loro **valore**.

- **Algoritmi Evolutivi:** Variante della ricerca beam stocastica basata su un comportamento simile alla selezione naturale:
 - Degli **stati**, tra questi vengono **trovati i migliori**, questi vengono **ricombinati tra di loro** per essere utilizzati e trovarne ancora dei migliori.
 - Quindi ogni **stato mappa** su un **individuo**.
 - Quindi un individuo può ad esempio essere rappresentato con una stringa.
 - Caratteristiche:
 - **Numero di genitori** $\rho = 2$ che si uniscono per generare la prole.
 - **Processo di selezione** genitori della prossima generazione, si seleziona con una **certa probabilità** rispetto ad un **valore** dipendente dalla funzione obiettivo, detto **fitness**.
 - **Procedura di ricombinazione:** Si seleziona un **punto di crossover**, quindi assumendo che i due **genitori** siano **due stringhe**, si definiscono dei **punti** in cui le stringhe vengono rispettivamente **tagliate e concatenate tra loro**.
 - **Mutazione:** A tempo di generazione della prole, i figli non saranno uguali a metà genitori ma esisterà una **mutazione** post-generazione, con **probabilità** data dal **tasso di mutazione**.
 - La **formazione della nuova generazione** potrebbe essere semplicemente la nuova prole, oppure **mantenere** tramite **elitismo** **genitori migliori rispetto a correnti figli**. Oppure anche tramite **abbattimento**, eliminando correnti figli sotto una certa **soglia di fitness**.

Un esempio di applicazione di questo tipo di algoritmo sul problema delle 8 regine:



Questi algoritmi genetici si applicano bene a specifici contesti e funziona bene su problemi i cui stati possono essere rappresentati in stringhe.

6.2 Ricerca Locale in Spazi Continui

Lo spazio non è più discreto ma descritto da **variabili continue** del tipo $x_1, \dots, x_n$.

L'**esplorazione** di questo tipo di spazi **può portare a fattori di ramificazione infiniti**, se utilizzassimo approcci visti fino ad ora.

- **Approccio di Spazi Continui:**
 - **Discretizzazione:** Si sovrappone una griglia allo spazio continuo.
 - **Gradiente Empirico:** Si limita il fattore di ramificazione mediante un campionamento casuale degli stati successivi, valutando ad esempio un valore perturbato di un x iniziale, tramite ad esempio una $f(x + \delta)$.
 - Se f è **continua e differenziabile** rispetto ad x , allora è possibile cercare **max e min** utilizzando il **gradiente**, che restituisce la **direzione di massima pendenza**. Quindi ha tutte le derivate parziali rispetto alle dimensioni della funzione f .

6.2.1 Utilizzo Hill Climbing in Spazi Continui con Gradiente

Possiamo applicare qualcosa come Hill Climbing tramite l'utilizzo del gradiente con la formula

$$x_{\text{new}} = x + \eta \nabla f(x)$$

dove η è una costante positiva detta **step size**.

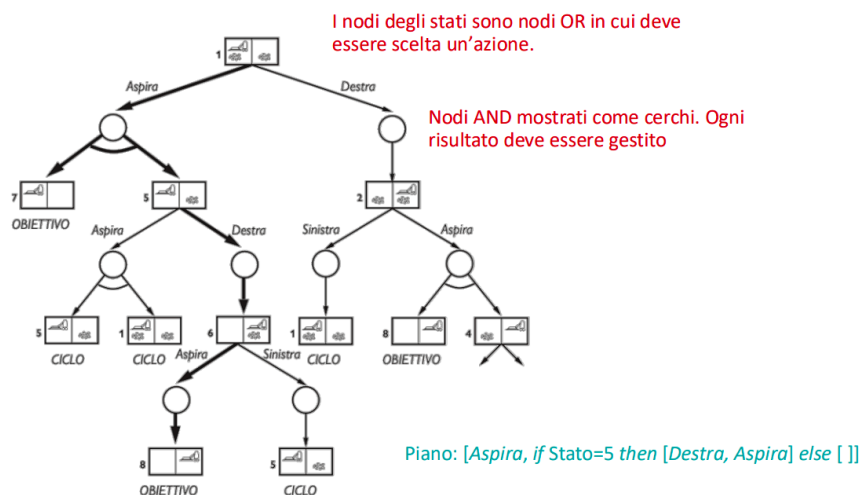
Questo meccanismo quindi ci fornisce la garanzia di crescita/descrescita.

6.3 Ricerca con Azioni non Deterministiche

Il piano generato in caso di determinismo è una sequenza di azioni ed eseguita senza imprevisti.

In questo caso **rilassiamo il determinismo**, un **azione** ha come **codominio un insieme di stati** e non uno stato, di conseguenza l'entità agentica dovrà **tramite percezioni che restringe lo spazio degli stati** acquisendo informazioni sullo stato corrente. Questo è detto quindi **stato-credenza**, **insieme** percepito dalle **percezioni dell'agente**.

Quindi dato ad esempio il problema dell'aspirapolvere, per trovare una soluzione non costruiremo un modello di transizione su cui costruiamo un percorso, ma definiamo un **albero AND-OR** che tiene conto delle condizioni su tutte le percezioni fatte durante il percorso e di questo ne **selezioniamo un sottoalbero**.



In questo caso il sottoalbero soluzione è quello rappresentato in grassetto.

7 PARTE II: LOGICA E AGENTI

8 — — — — Lezione 6 - Agenti Basati su Conoscenza - 05/03/2026

Riferimento AIMA Cap 7.1 - 7.3

8.1 Agenti Knowledge Based KB e Dichiarativo vs Procedurale

Si aumentano le capacità degli agenti di visualizzare ed avere la rappresentazione di mondi complessi e astratti. Questo permette definizione di agenti detti **Knowledge Based KB**:

- Si definisce quindi la **necessità di rappresentare la conoscenza** che siano espressivi e con **capacità di inferenza**.

Approccio Dichiarativo vs Procedurale:

- **Dichiarativo:** Definito su quello che il modello deve sapere, questo viene definito tramite una funzione TELL.
 - Si inizia da una conoscenza di base vuota e si aggiungono mano mano formule di conoscenza.
- **Procedurale:** Definito il programma che implementa il processo decisionale, una volta per tutte.

Formulazione Agenti Knowledge Based:

Insieme di enunciati espressi in un linguaggio di rappresentazione, si interagisce con questa conoscenza tramite un'interfaccia funzionale Tell-Ask:

- Tell: per aggiungere nuova Knowledge.
- Ask: per interrogare la Knowledge.
- Retract: per eliminare Knowledge.

8.2 Conseguenza Logica e Knowledge Base (KB)

Definiamo cosa sia una **conseguenza logica**, elencando le **differenze** tra KB ed una normale **base di dati**.

Definizione di Conseguenza Logica:

Data una **base di conoscenza KB**, contenente una **rappresentazione dei fatti** che si **ritengono veri**, come dedurre che un certo fatto α è vero di conseguenza?

$$KB \models \alpha$$

Tutte le risposte α devono discendere direttamente dalla KB, quindi $KB \models \alpha$.

Un programma agente sarà:

```
function Agente-KB (percezione) returns un'azione
  persistent: KB, una base di conoscenza
             t, un contatore, inizialmente a 0, che indica il tempo

  TELL(KB, Costruisci-Formula-Percezione(percezione, t ))
  azione = ASK(KB, Costruisci-Query-Azione( t ))
  TELL(KB, Costruisci-Formula-Azione(azione, t ))
  t = t + 1
  return azione
```

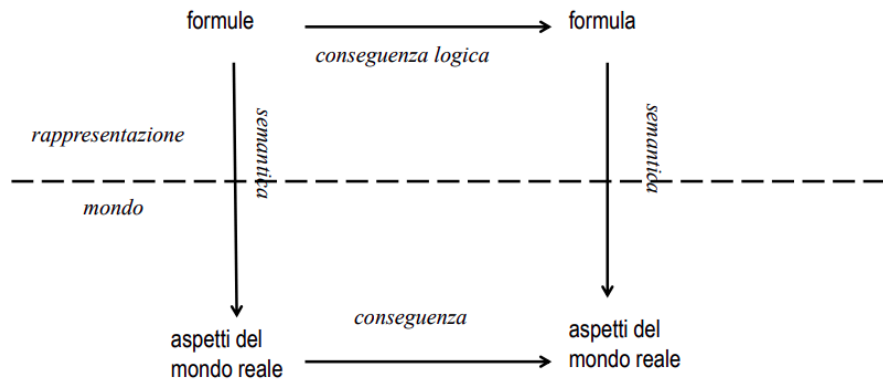
Knowledge Base vs Base di Dati:

- La Base di Conoscenza permette di effettuare inferenza sulle informazioni, a differenza della base di dati che contiene solo fatti specifici, immagazzinati per eventuale recupero.
- Su questo si basa anche un **trade-off** per cui il linguaggio più è espressivo e **meno efficiente** sarà il **meccanismo inferenziale**.

8.3 Logica

Si basa su tre elementi principali:

- **Sintassi:** Definire quando un'espressione sia o meno ben formata.
- **Semantica:** Esprime il significato della formula.
- **Meccanismo Inferenziale:** Permette di inferire nuovi fatti, dalla base di conoscenza.



8.4 Agenti Logici: Calcolo e Logica Proposizionale

AIMA Cap 7.3 e 7.4

- **Sintassi linguaggio Proposizionale:** Definite tramite una BNF:

$$\begin{aligned} \text{formula} &\rightarrow \text{formulaAtomica} \mid \text{formulaComplessa} \\ \text{formulaAtomica} &\rightarrow \text{True} \mid \text{False} \mid \text{simbolo} \\ \text{simbolo} &\rightarrow P \mid Q \mid R \mid \dots \\ \text{formulaComplessa} &\rightarrow \neg \text{formula} \\ &\text{and (coniunzione)} \mid (\text{formula} \wedge \text{formula}) \\ &\text{or (disgiunzione)} \mid (\text{formula} \vee \text{formula}) \\ &\text{implicazione} \mid (\text{formula} \Rightarrow \text{formula}) \\ &\text{se e solo se} \mid (\text{formula} \Leftrightarrow \text{formula}) \end{aligned}$$

- **Semantica linguaggio Proposizionale:** Si definisce un'interpretazione specificando il valore di verità per ciascun simbolo proposizionale.
- **Modello di una Formula:** Interpretazione che rende vera la formula.

8.4.1 Definizione Conseguenza Logica

Una formula α è conseguenza logica di un insieme di formule KB se e solo se in ogni modello di KB, anche α è vera, ossia $KB \models \alpha$. Quindi

$$KB \models \alpha \Leftrightarrow M(KB) \subseteq M(\alpha)$$

8.4.2 Definizione Model Checking

Un modo per determinare la **conseguenza logica**:

- Enumerando i modelli
- Mostrando che la formula α deve valere in tutti i modelli in cui è vera la KB.
- Si usa per eseguire inferenza logica.

8.5 Calcolo Proposizionale negli Agenti Logici

AIMA Cap 7.6

La **conseguenza logica** non viene definita solo tramite **model checking** ma **anche tramite dimostrazione**:

- Applicando regole di inferenza direttamente alle formule della KB.
- Costruendo una dimostrazione della formula desiderata senza consultare i modelli.

Tutto questo è basato su **tre principi**, ossia:

- **Equivalenza logica**: Due formule α e β sono equivalenti se sono vere nello stesso insieme di modelli.
- **Validità**: Una formula α è valida se e solo se è vera in ogni interpretazione, queste formule sono dette anche tautologie.
 - Si basa sul teorema di deduzione, ossia che date due formule α e β vale che

$$\alpha \models \beta \Leftrightarrow (\alpha \Rightarrow \beta) \text{ è valida}$$

- **Soddisfacibilità**: Una formula α è soddisfacibile se e solo se esiste almeno un interpretazione in cui α è vera.

Definizione Dimostrazione per Assurdo:

- Per il Th. di deduzione vale che $\alpha \models \beta \Leftrightarrow (\alpha \Rightarrow \beta)$ è valida.
- $\alpha \models \beta \Leftrightarrow \neg(\alpha \Rightarrow \beta)$ è insoddisfacibile.
- $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \equiv (\alpha \wedge \neg\beta)$
- Quindi $\alpha \models \beta \Leftrightarrow (\alpha \wedge \neg\beta)$ è insoddisfacibile.

Inferenza per PROP:

- **Model Checking**:
 - Forma di inferenza che fa riferimento alla definizione di conseguenza logica, enumerando i possibili modelli.
 - Si utilizza una tecnica con tabelle di (TV).
- **Algoritmi per la Soddisfacibilità (SAT)**:
 - $KB \models \alpha \Leftrightarrow (KB \wedge \neg\alpha)$ è insoddisfacibile, dal teorema di refutazione.
 - La conseguenza logica può essere ricondotta a un problema SAT.

8.5.1 Model Checking e Algoritmo TV-Consegue

Consiste in:

- Enumerare tutti i possibili modelli di KB, tramite l'utilizzo della tabella di verità.
- Verificare che in essi la formula α sia vera.

Algoritmo TV-Consegue: Segue questi passi, cercando di capire se $KB \models \alpha$:

- Enumera tutte le possibili interpretazioni di KB
 - Su k simboli produrrà quindi 2^k possibili interpretazioni.
- Per ciascuna interpretazione
 - Se non soddisfa KB, allora viene marcata come OK.
 - Se soddisfa KB, si controlla che soddisfi anche α .
- Se si trova anche solo una interpretazione che soddisfa KB e non α la risposta sarà NO.

8.5.2 Algoritmi per la Soddisfacibilità (SAT)

Basata sulla forma a clausole in **forma normale congiuntiva (CNF)**, ossia:

$$(A \vee B) \wedge (\neg B \vee C \vee D) \wedge (\neg A \vee F)$$

Non è restrittiva, è sempre possibile ottenerla con trasformazioni che **preservano l'equivalenza logica**. Rappresentata quindi come insieme di letterali del tipo

$$\{A, B\}\{\neg B, C, D\}\{\neg A, F\}$$

8.5.2.1 Trasformazione in Forma Normale Congiuntiva (CNF)

Si basa sui seguenti passi:

- **Eliminazione del $\Leftrightarrow$:** $(A \Leftrightarrow B) \equiv (A \Rightarrow B) \wedge (B \Rightarrow A)$
- **Eliminazione del $\Rightarrow$:** $(A \Rightarrow B) \equiv (\neg A \vee B)$
- **Negazioni all'interno (De Morgan):**
 - $\neg(A \vee B) \equiv (\neg A \wedge \neg B)$
 - $\neg(A \wedge B) \equiv (\neg A \vee \neg B)$
- **Distribuzione di $\vee$ su $\wedge$:** $(A \vee (B \wedge C)) \equiv (A \vee B) \wedge (A \vee C)$

8.5.2.2 Algoritmo DPLL

Algoritmo **completo e che termina sempre** basato sui seguenti passi:

- Si parte da una KB in **forma a clausole**
 - Prende in input una **formula CNF**, quindi **insieme di clausole**.
- Si effettua un **enumerazione ricorsiva** in profondità di tutte le possibili interpretazioni alla ricerca di un modello.
- Si basa su **tre ottimizzazioni** rispetto a TV-Consegue:
 - **Terminazione Anticipata:**
 - Si può decidere sulla base anche di interpretazioni parziali.
 - Se anche una sola clausola è falsa, l'interpretazione non può essere un modello dell'insieme di clausole.
 - **Euristica di Simboli Puri:**
 - Simboli che appaiono con lo stesso segno in tutte le clausole, possono quindi essere direttamente sostituiti con true o false.
 - **Clausole Unitarie:** Una clausola con un solo letterale non assegnato. Conviene quindi assegnare prima valori al letterale in clausole unitarie.

8.5.2.3 Metodi Locali per SAT - WalkSat

Gli stati sono interpretazioni, l'obiettivo è un assegnamento che soddisfa tutte le clausole.

- Si **flippano** gli stati per **spostarsi** tra il numero di **clausole soddisfatte**.

Algoritmi per Metodi Locali SAT

Abbiamo già visto Hill Climbing e Simulated Annealing, ma in questo contesto si utilizza il WalkSat.

WalkSat Ad ogni passo si eseguono queste operazioni:

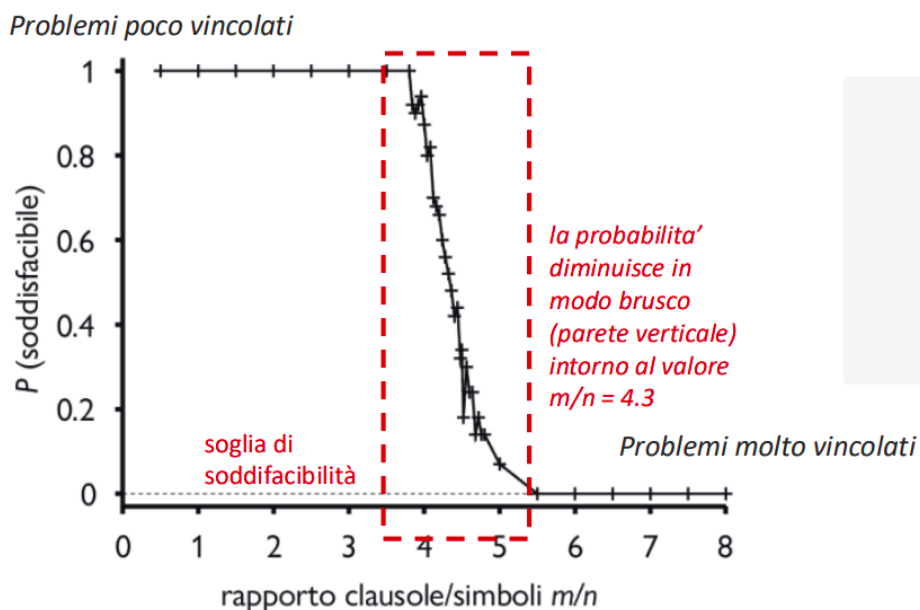
- Sceglie a caso una clausola non ancora soddisfatta
- Individua un simbolo da modificare flip, scegliendo con probabilità $p = 0.5$ tra:
 - Scegliere un simbolo a caso, secondo il **random walk**.
 - Scegliere quello che rende più clausole soddisfatte, secondo il **passo di ottimizzazione**.
- L'algoritmo si ferma dopo un certo numero di flip predefinito.

Caratteristiche WalkSat: Va bene per cercare un modello sapendo che esiste, ma se l'insieme delle clausole fosse insoddisfacibile allora non terminerebbe mai. Quindi non può essere utilizzato per trovare insoddisfacibilità.

Quindi utilizziamo questo algoritmo per poter dire **che non possiamo dimostrare che vale la conseguenza logica**.

Problemi SAT difficili:

Si basa sul rapporto $\frac{m}{n}$ dove m è il numero di clausole ed n numero di simboli, quindi in qualche senso grado di libertà. Quindi maggiore è $\frac{m}{n}$ e più vincolato è il problema.



Nella zona evidenziata dei problemi difficili ci comporta meglio Walksat, ma se sono in una zona non difficile allora utilizzo DPLL e non Walksat.

9 — — — — — Lezione 7 - Calcolo Proposizionale (PROP) - 06/03/2026

Rif AIMA 7.5, 7.6

9.1 Inferenza come deduzione

Per determinare se $KB \models A$ è usare un **sistema di deduzione**.

- Definiamo la derivabilità con $KB \vdash A$
- Si specificano **regole di inferenza**.
 - Si dovrebbero derivare **solo** formule che sono conseguenza logica.
 - Si dovrebbero derivare **tutte** le formule che sono conseguenza logica.

9.1.1 Correttezza e Completezza

- **Correttezza:** Se $KB \vdash \alpha$ allora $KB \models \alpha$ quindi
 - Tutto ciò che è derivabile è conseguenza logica.
 - Le regole preservano la verità.
- **Completezza:** Se $KB \models \alpha$ allora $KB \vdash \alpha$ quindi
 - Tutto ciò che è conseguenza logica è ottenibile tramite il sistema deduttivo.

9.1.2 Dimostrazione come Ricerca (Direzione, Strategia)

Una dimostrazione diventa una visita di spazi di stati, tramite una procedura che specifica:

- **Direzione della ricerca:** Nella dimostrazione di teorema si preferisce andare in direzione opposta.
- **Strategia della ricerca:** Un algoritmo che sia **completo** ed **efficiente**.

Utilizzo in Composizione di Th. Refutazione e Th. Risoluzione

- **Th. Refutazione:** Offre un modo alternativo per procedere

$$KB \models \alpha \Leftrightarrow (KB \cup \{\neg\alpha\}) \text{ è insoddisfacibile}$$

- **Th. Risoluzione:** Vale che

$$KB \text{ insoddisfacibile} \Leftrightarrow KB \vdash_{\text{res}} \{\}$$

Quindi volendo **istanziare** un **utilizzo** di questo metodo con $\alpha = \{A, \neg A\}$

- Per il Th. Refutazione vale che

$$KB \cup FC(\neg(A \vee \neg A))$$

- Quindi passando alla **formulazione a clause** ed applicando **Th. Risoluzione**

$$KB \cup \{A\} \cup \{\neg A\} \vdash_{\text{res}} \{\}$$

- In conclusione vale che $KB \models \{A, \neg A\}$

Sintetizzando

Abbiamo tre metodologie: Tavole verità, SAT e Refutazione.

10 — — — — Lezione 8 - Logica del Prim'Ordine (FOL) - 10/03/2026

Rif. AIMA 8.1 8.2 8.3

In questa lezione il focus è sulla **sintassi** e sulla **semantica** della FOL.

Si inizia a concettualizzare, rappresentando oggetti tramite **simboli** o **mediante funzioni**.

Si definiscono quindi **relazioni** tra oggetti e **proprietà** di oggetti, viste come **relazioni unarie**.

10.1 Sintassi della FOL

Gli elementi sintattici di base sono i simboli usati per indicare gli oggetti, le relazioni e le funzioni, quindi:

- Simbolo di **costante**
- Simbolo di **predicato**
- Simbolo di **funzione**

Termini: Espressione logica che si riferisce ad un oggetto.

Termine $\rightarrow$ Costante | Variabile | Funzione(Termine)

Uguaglianza: Si utilizza per definire che due termini riferiscono allo stesso oggetto

Padre(Claudio) = Giampiero

Formule Atomiche/Complesse:

- **Atomica:**

FormulaAtomica $\rightarrow$ True | False | Termine = Termine

- **Complessa:**

Formula $\rightarrow$ FormulaAtomica | Formula Connettivo Formula |
| Quantificatore Variabile Formula |
| $\neg$ Formula | (Formula)

Quantificatori:

- **Quantificazione Esistenziale/Universale:**

- **Universale:** La relazione si applica a tutti gli elementi del dominio.
- **Esistenziale:** La relazione si applica ad almeno un elemento del dominio.

- **Scope dei Quantificatori:**

- **Esempio:** $\forall x(\exists y P(x, y))$ tutto è scope del $\forall$, mentre solo $P(x, y)$ appartiene allo scope di $\exists$.
- **Binding:** Se una variabile è in uno scope di quantificatore allora si dice che la variabile è legata, altrimenti è libera.

Precedenze tra Operatori: Seguono quest'ordine

$=, \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow, \forall, \exists$

10.2 Semantica della FOL

Le interpretazioni sono più complesse rispetto alla PROP, vanno mappati i simboli sul dominio modellato.

- **Universo del Discorso:** Un insieme non vuoto di elementi che rappresenta l'insieme di tutti gli oggetti che si prendono in considerazione.
- **Assegnazione di Valori:**
 - **Costanti:** Assegnate ad un elemento specifico dell'universo.
 - **Variabili:** Si possono assegnare ad elementi del dominio tramite funzioni di Assegnazione
 - **Funzioni:** Si mappano una sequenza di elementi del dominio su un singolo elemento.
 - **Predicati:** Assegnata una relazione sull'universo di discorso.
- **Verità di Formule:**
 - **Formula Atomica:** Una formula atomica è detta vera se l'interpretazione assegna ai suoi termini una sequenza che rientra nella relazione specificata dal predicato.
 - **Formula Complessa:** Sono valutate sulla base dei loro componenti usando il significato dei connettivi logici e dei quantificatori.
- **Interpretazione:** Stabilisce una **corrispondenza** precisa tra **elementi atomici** del **linguaggio** ed **elementi** della **concettualizzazione**.

Si basa sulla **composizionalità**, quindi predicati e funzioni vengono utilizzati tra loro in composizione.

10.2.1 Semantica degli Operatori di Quantificazione

Esiste una relazione tra $\forall$ e $\exists$ ossia:

$$\forall x \neg P(x) \equiv \neg \exists x P(x)$$

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

$$\forall x P(x) \equiv \neg \exists x \neg P(x)$$

$$\neg \forall x \neg P(x) \equiv \exists x P(x)$$

10.3 Interazione con Knowledge Base KB in FOL

Grazie alla FOL possiamo definire **asserzioni** ed **interrogazioni** ad una **KB** tramite rispettivamente le funzioni TELL e ASK.

```
TELL(KB, Re(Giovanni)) //asserzione
ASK(KB, exists x Persona(x)) //query
```

10.4 Inferenza nella FOL

Rif. AIMA 9.1 9.2

Dato che già conosciamo come inferire su PROP, allora possiamo convertire KB della FOL in PROP ed applicare metodi già visti, handlando i quantificatori che non esistono in PROP.

10.4.1 Istanziamento Universale/Esistenziale

Istanziamento Universale - Gestione $\forall$: Sostituiamo le variabili quantificate universalmente con termini ground, quindi una $\text{SUBST}(\{x/g\}, A)$, quindi sostituisco x con g nella formula A e g è detto termine ground e $A[x/g]$ è il risultato della sostituzione.

Istanziamento Esistenziale con Skolem - Gestione $\exists$: Esistono **due casi**:

- Se $\exists$ non compare nello scope di $\forall$ allora k è una costante nuova.
- Altrimenti va introdotta una funzione di Skolem nelle variabili quantificate universalmente

$$\exists x \text{ Padre}(x, G) \text{ diventa } \text{Padre}(K, G)$$

$$\forall x \exists y \text{ Padre}(x, y) \text{ diventa } \forall x \text{ Padre}(x, p(x))$$

10.4.2 Grounding e Teorema di Herbrand

Grounding: Processo di proposizionalizzazione, segue questi passi:

- Si creano tante istanze delle formule quantificate universalmente quanti sono gli oggetti menzionati.
- **Eliminare** i quantificatori **esistenziali skolemizzando**.
- **Sostituire** le **formule atomiche** ground con **simboli proposizionali**.

Questo può portare problematiche, infatti **costanti** sono **finite** ma il **numero di istanze** è **infinito**.

Th. di Herbrand Se $KB \models \alpha$ c'è una dimostrazione che coinvolge solo un sottoinsieme finito della KB proposizionalizzata.

10.4.3 Forma a Clausole della FOL

Definiamo che una clausola è un insieme di letterali che rappresenta la loro disgiunzione, quindi lasciando la possibilità di rappresentare KB in forma a clausole.

Teorema Per ogni formula chiusa α della FOL è possibile trovare in maniera effettiva un insieme di clausole $\text{FC}(\alpha)$ che è soddisfacibile se e solo se α è soddisfacibile.

10.4.4 Trasformazione in FC della FOL

Come nella PROP, anche nella FOL esiste una procedura per convertire formule in forma a clausole:

- **Eliminazione delle implicazioni.**
- **Negazioni all'interno.**
- **Standardizzazione delle variabili:** Ogni quantificatore usa variabili diverse, anche se non abbiamo conflitti in scope.
- **Skolemizzazione:** Eliminazione della quantificazione esistenziale, tramite costante di Skolem se non in scope di universale, e tramite funzione di Skolem se in scope di universale.
- **Eliminazione Quantificazione Universale:** Si portano fuori tutti i quantificatori universali usando la convenzione che tutte le variabili libere sono quantificate universalmente.
- **Forma Normale Congiuntiva:** Congiunzione di disgiunzione di letterali.

- **Notazione a Clausole.**
- **Separazione delle Variabili:** Clausole diverse, variabili diverse.

10.4.5 Unificazione ed Algoritmo di Unificazione

Operazione mediante la quale si determina se due espressioni possono essere rese identiche mediante una sostituzione di termini a variabili.

Questo ci serve per **identificare un simil** $A, \neg A$ come nel passo di risoluzione in PROP, ma qui non è così semplice identificare $A, \neg A$. Per questo motivo esiste questo **procedimento di unificazione**.

Si basa su una **sostituzione**, ossia un insieme finito di associazioni tra variabili e termini in cui ogni variabile compare una sola volta sulla sinistra. Ad esempio potrebbe essere della forma

$$\sigma = \{x_1/A, x_2/f(f_4), x_3/B\}$$

Quindi utilizzandola nella funzione $\text{subst}()$:

$$\text{subst}(\sigma, A)$$

Espressioni Unificabili Se esiste una sostituzione che rende le espressioni **identiche** oppure **fail**. Si cerca in particolare non una semplice sostituzione ma la **Most General Unifier MGU**, che viene definita con il numero più piccolo possibile di sostituzioni.

Algoritmo di Unificazione L'algoritmo prende in input due espressioni p, q e restituisce un **MGU** θ se esiste:

- $\text{UNIFY}(p, q) = \theta$ tale che $\text{SUBST}(\theta, p) = \text{SUBST}(p, \theta)$
- altrimenti fail

L'algoritmo esplora in parallelo le due espressioni e costruisce l'unificatore.

Appena trova espressioni non unificabili fallisce.

Una causa di fallimento sono sostituzioni del tipo $x = f(x)$, e questo controllo è detto **occur check**.

10.5 Inferenza e Completezza del Metodo di Risoluzione

Si prosegue per refutazione anche in questo caso, dato che questo metodo è completo, quindi come visto in PROP quindi

$$\text{KB} \cup \{\neg\alpha\} \text{ è insoddisfacibile} \Leftrightarrow \text{KB} \models \alpha$$

quindi come già visto in PROP, applichiamo il teorema per cui

$$\text{KB è insoddisfacibile} \Leftrightarrow \text{KB} \vdash \{\}$$

10.6 Cenni di Sistemi a Regole

Rif Cap 9.3, 9.4

10.6.1 Clausole di Horn

Una clausola di Horn è una **disgiunzione di letterali** che contiene **al massimo un letterale positivo**.

Questo ci permette in modo efficiente di rappresentare la conoscenza in FOL.

Basato sul concetto per cui la parte a sinistra di un'implicazione, se è congiunzione, post trasformazione dell'implicazione diventa una disgiunzione.

Quindi definendo formalmente le **Clausole di Horn Definite**, quindi con esattamente un letterale positivo:

$$\{Q, \neg P_1, \dots, \neg P_k\} \text{ dato } (k \geq 0)$$

Quindi una KB può essere espressa in regole come

$$P_1, \dots, P_k \Rightarrow Q \text{ (regola)}$$
$$Q \text{ (fatto)}$$

In questo modo si definiscono meccanismi inferenziali sono molto più semplici, senza rinunciare alla completezza del metodo, anche se non corrisponde esattamente alla FOL, è una sorta di suo sottoinsieme.

10.6.2 Backward/Forward Chaining

- **Forward Chaining:** Inizia dalle conclusioni, ossia l'obiettivo da dimostrare, e lavora a ritroso per trovare le premesse. Approccio utilizzato spesso nella programmazione logica, come nel PROLOG.
- **Backward Chaining:** Si inizia dalle premesse e si procede verso le conclusioni. Utilizzato nei sistemi esperti per dedurre conclusioni partendo da un insieme di fatti.

10.6.3 Programmazione Logica

Basata su programmi logici, ossia KB costituiti da clausole di Horn, definite come **fatti e regole**, quindi una sintassi alternativa.

Le variabili sono indicate con lettere maiuscole, le costanti con lettere minuscole.

Quindi un goal del tipo

$$B_1 \wedge B_2 \dots B_{n-1} \wedge B_n \text{ viene espresso come } :- B_1, B_2 \dots B_{n-1}, B_n$$

10.6.3.1 Risoluzione Selection Linear Definite-clauses (SLD)

Strategia ordinata e completa per clausole di Horn, si parte da un programma P definito in PL e dato un goal G si costruisce l'albero di risoluzione.

Partendo quindi da un insieme di goals del tipo $G_1, G_2 \dots G_{n-1}, G_n$ proviamo ad unificare, dal primo in poi, con la testa di una delle regole date.

11 PARTE III: MACHINE LEARNING

12 — — — — — Lezione 9 - Introduzione al Machine Learning - 12/03/2026

Quando bisogna utilizzare soluzioni basate sul Machine Learning?

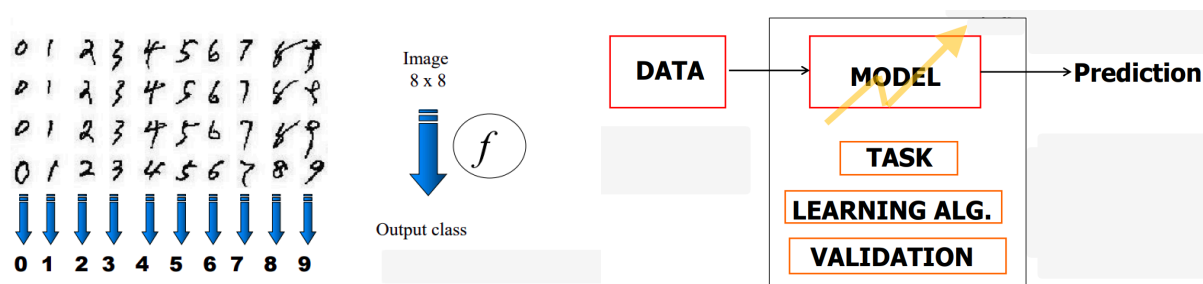
- In caso di ambienti dinamici.
- Ambienti in cui non è stata formalizzata la teoria.
- Bisogna che esistano dataset su cui basare l'allenamento del modello.

Quindi se la priorità fosse quella di avere una precisione del 100% allora potrebbe essere il caso di non utilizzare il Machine Learning.

12.1 Overview di un ML

L'idea è quella di una approssimazione di funzioni sconosciute partendo da esempi dati.

Un esempio potrebbe essere il riconoscimento di caratteri data una matrice di pixel:



L'obiettivo è quindi quello di approssimare la funzione f .

12.2 Supervised Learning e Classification/Regression

Definizione Supervised Learning

- **Dato** un insieme di esempi di training (input, output) $= (x, d)$, dove d è una label, per definire la funzione f .
- Si **trova** una buona approssimazione della f
- **Classificazione**: $f(x)$ ritorna la classe corretta per x , quindi esce in un insieme di classi discreto del tipo $\{1, 2, \dots, k\}$
- **Regressione**: $f(x)$ che ritorna reali in output, quindi rispetto al tipo precedente varia il codominio.

12.3 Unsupervised Learning

Il Training Set si compone di elementi senza label, quindi (x) , basato sul **clustering**, quindi **partizione dei dati in sottoinsiemi**.

12.4 Modello

Cerca di catturare la relazione tra i dati e definisce la classe delle funzioni che il modello può implementare:

- **Esempio di Training:** Un esempio della forma $(x, f(x) + \text{noise})$, dove x è un vettore di features.
- **Funzione Target:** La reale funzione f .
- **Ipotesi :** Una funzione h proposta che si ipotizza essere simile alla f target. Un'espressione in un linguaggio dato che descrive la relazione tra i dati.
- **Spazio Ipotesi :** Lo spazio di tutte le ipotesi che dovrebbe corrispondere all'output dell'algoritmo di apprendimento.

12.4.1 Tipi di Modelli - (Lineari, Simbolici, Probabilistici, Instance Based)

- **Lineari:** Basati sulla rappresentazione di una funzione h parametrica e successivamente istanziata, quindi **rappresentazione continua** di uno **spazio di ipotesi**:

$$h_w(x) = w_1x + w_0 \quad h_w(x) = 0.232x + 246$$

- **Simbolici:** Lo spazio di ipotesi è rappresentato in modo discreto, ad esempio:

```
if (x1 = 0 && x2 = 1)
  then h(x) = 1
  else h(x) = 0
```

- **Probabilistici:** Basati su una stima probabilistica $p(x, y)$.
- **Instance Based:** Basati sulla predizione del valore medio di y dei vicini.

Questo ci permette di definire come viene formulato in formule, ma non come questi parametri vengano scelti. Questo sta all'**algoritmo di learning scelto**.

12.5 Algoritmo di Learning

Basati sulla **ricerca** nello **spazio di ipotesi** della **migliore** tra tutte le **ipotesi**.

- La **migliore approssimazione** della **funzione target sconosciuta**, quindi cercare la h con il **minor errore**.

12.6 Generalizzazione del ML

La **bontà di un modello non** si misura **solo sul fitting** generato sul dataset e la funzione di approssimazione, ma **soprattutto** sulla **generalizzazione**, ossia sulla **capacità di rispondere** in maniera appropriata a **nuovi casi**, non appartenenti al dataset di partenza.

Def. Errore di Generalizzazione L'errore che misura quanto accuratamente il modello predice nuovi sample di dati, definendo quindi anche misure di error e loss.

- **Fasi della Generalizzazione**
 - **Fase di Learning (training e fitting):** Si costruisce il modello dai dati conosciuti, tramite fase di training.
 - **Fase di Predizione (testing):** Preso un nuovo input x' , questo viene dato al modello che computa una risposta sulla base del nuovo input e questo viene valutato in base alla capacità di generalizzazione.

13 — — — — — Lezione 10 - Concept Learning - 17/03/2026

Rif. Slide IIA-26-ML-concept-learning-v0.1.pdf

Il **Concept Learning** si basa su inferenza di una funzione booleana partendo da esempi di training.

Il **primo modello** per che possiamo immaginare è quello basato su **lookup table**, che **non scala per nulla**, in cui dati gli input guardiamo la tabella e rispondiamo rispetto alla sua entry.

13.1 Regole Congiuntive

Nel contesto della costruzione di ipotesi, quante regole congiuntive (di letterali in congiunzione) possiamo definire? Utilizziamo le cardinalità, quindi semplici regole congiuntive possono essere $|H| = 2^n$, con n lunghezza di una l_i istanza di ipotesi h_i .

13.2 Rappresentazione di Ipotesi

Un ipotesi h è una **congiunzione di vincoli su attributi**.

Nel Mitchell si usa la corrente notazione per esprimere vincoli sugli attributi:

	Sky	Temp	Humid	Wind	Water	Forecast
<	Sunny	?	?	Strong	?	Same>

Quindi riferiremo ad una composizione più specifica ed una più generale, ossia:

<	∅	∅	∅	∅	∅	∅	>	Most specific
<	?	?	?	?	?	?	>	Most general

13.2.1 Definizione di Prototypical Concept Learning Task

Siano **dati**:

- Gli **attributi** $X = \{\text{Sky, Temp, Humidity, Wind ...}\}$ definiamo:
- Una **funzione target** $c : \text{EnjoySport } X \rightarrow \{0, 1\}$
- Un **ipotesi** H , ossia congiunzione di un insieme finito di letterali, quindi ad esempio

< Sunny ? ? Strong Same >

- Degli **esempi di training** D , ossia esempi positivi e negativi della funzione target.

Si **trovi**:

- Un **ipotesi** h in H tale per cui vale che $h(x) = c(x)$ per ogni $x \in X$.

Il concetto di learning si basa quindi sulla ricerca nello spazio delle ipotesi H .

13.2.2 Assunzione sulla Inductive Learning Hypothesis

Assunzione: Tutta questa lezione si basa su un'assunzione della Inductive Learning Hypothesis, ossia che tutte le ipotesi che approssimano bene la funzione target per gli esempi di training lo faranno anche per la funzione target degli esempi non osservati. Questo verrà dimostrato, ma per adesso viene assunto.

Ordinamento su Specificità di Ipotesi: Un ipotesi h_1 può essere più larga rispetto ad una h_2 mettendo meno vincoli sugli attributi, questo permette una definizione di ordinamento parziale tra le ipotesi.

Formalmente: Date h_j, h_k due funzioni booleane definite su X . Se h_j è più generale o uguale h_k , quindi vale che $h_j \geq h_k$ se e solo se

$$\forall x \in X : [(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$$

13.2.3 Algoritmo Find-S

Funzionamento a Passi:

- Si inizializza h con l'ipotesi più specifica in H .
- Per ogni istanza di training positiva:
 - Per ogni attributo a_i in h
 - Se l'attributo è già soddisfatto in h da x allora non succede nulla.
 - Altrimenti sostituisci a_i in h con il vincolo più generale possibile soddisfatto da x e rimuovi da h i letterali che non soddisfano x .

Quindi si parte dalla più specifica e si cerca di scendere verso una più generica che rispetti gli input passati.

Proprietà Find-S:

- Spazio delle ipotesi descritto da congiunzioni di attributi.
- Find-S tira fuori l'ipotesi consistente agli esempi positivi più specifica possibile.
- Quindi c attesa sarà sicuramente $c \geq H$

Problemi Find-S:

- Non sappiamo se si converge al target.
- Non possiamo affermare se i training data siano inconsistenti, dato che ignoriamo i casi negativi.

13.2.4 Definizione di Version Spaces

Il **version space** $VS_{H,D}$ basato su uno **spazio di ipotesi** H ed un **training set** D corrisponde al sottoinsieme delle ipotesi H **consistenti** con tutti gli esempi di training, ossia:

$$VS_{H,D} = \{h \in H \mid \text{Consistent}(h, D)\}$$

13.2.5 Version Space tramite General/Specific Boundary (con Th.)

- **General Boundary:** G del **version space** $VS_{H,D}$ rappresenta l'insieme dei membri più generici, tenendo conto di H consistente con D .
- **Specific Boundary:** S del **version space** $VS_{H,D}$ rappresenta l'insieme dei membri più specifici, tenendo conto di H consistente con D .
- **Teorema:** Ogni membro del version space si trova tra i due boundaries:

$$VS_{H,D} = \{h \in H \mid (\exists s \in S)(\exists g \in G)(g \geq h \geq s)\}$$

dove $x \geq y$ significa che x è **più generale o uguale** ad y .

13.2.6 Algoritmo Candidate Elimination

Si definiscono G, S rispettivamente **insieme ipotesi più generale e più specifica** in H .

Allora **per ogni** esempio di training $d = (x, c(x))$:

- Se d è un **esempio positivo**:
 1. Rimuovi da G qualsiasi ipotesi che risulta essere inconsistente con l'esempio d :
 2. Per ogni singola ipotesi $s \in S$ che non è consistente con d :
 - Rimuovi s da S .
 - Aggiungi ad S tutte le minime generalizzazioni di h di s tali per cui valgono queste due condizioni:
 - h resta consistente con d .
 - Dei membri di G sono più generali rispetto ad h .
- Se d è un **esempio negativo**:
 1. Rimuovi da S qualsiasi ipotesi che risulta essere inconsistente con l'esempio d :
 2. Per ogni singola ipotesi $g \in G$ che non è consistente con d :
 - Rimuovi g da G .
 - Aggiungi a G tutte le minime specializzazioni di h di g tali per cui valgono queste due condizioni:
 - h resta consistente con d .
 - Dei membri di S sono più generali rispetto ad h .
- Infine pulisci da S qualsiasi ipotesi che risulta essere più generale di altre ipotesi in S .

13.3 Inductive Bias

Il nostro spazio di ipotesi non può rappresentare l'operatore $\vee$.

Bias: Assumiamo che lo **spazio d'ipotesi H contenga il concetto target c** . In altre parole c può essere descritto come **coniunzione di letterali**.

Learner Senza Bias: Se non avessimo questo bias di riferimento allora assumendo di avere un esempio positivo (x_1, x_2, x_3) ed uno negativo (x_4, x_5) allora staremmo definendo

$$S = \{x_1 \vee x_2 \vee x_3\} \quad G = \{\neg(x_4 \vee x_5)\}$$

Grazie a questo esempio possiamo quindi definire che un learner **senza bias non è in grado di generalizzare**.

Questo perchè avremo modo solo di valutare rispetto a cosa si ha esattamente in S ed in G e non nel mezzo.

13.3.1 Definizione Formale

Dati:

- Un algoritmo di learning L .
- Delle istanze X ed un concetto target c .
- Degli esempi di training $D_c = \{(x, c(x))\}$.
- Sia $L(x_i, D_c)$ la classificazione assegnata all'istanza x_i da L dopo allenamento su D_c .

Si definisce:

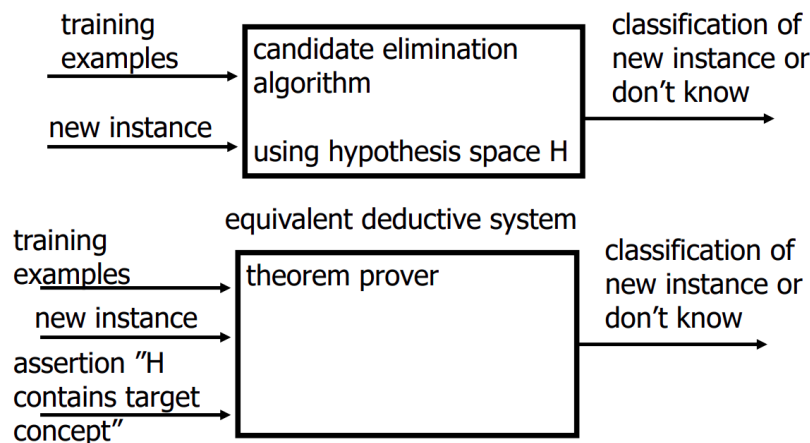
- Il bias induttivo su L è l'insieme minimale di asserzioni B tali per cui per ogni concetto target c e corrispondenti dati di training D_c vale che:

$$(\forall x_i \in X)[B \wedge D_c \wedge x_i] \vdash L(x_i, D_c)$$

dove con un generico $A \vdash B$ si intende che da A possiamo logicamente dedurre B .

13.3.2 Sistemi Induttivi ed Equivalenti Deduttivi e Riassunto Learners

Mostriamo degli esempi grafici rispettivamente di un sistema induttivo e di un equivalente deduttivo.



Tre Learners Corrente Lezione e Caratteristiche:

- **Lookup Table:** Osserva e salva esempi, classifica x se e solo se matcha con un esempio già osservato.
- **Version Space Candidate Elimination Algorithm:** Si basa sull'utilizzo del **bias induttivo**, si assume quindi che lo spazio delle ipotesi H contenga il concetto target c come congiunzione di attributi.
- **Find-S:** Basato sul **language bias** dato dall'operatore $\wedge$ ed il **search bias** data la preferenza sull'ipotesi più specifica.
 - Lo spazio delle ipotesi contiene il concetto target e tutte le istanze sono negative finché l'opposto non è implicato da altra knowledge, vista come esempi positivi.

14 — — — — Lezione 11 - Modelli Lineari I - 19/03/2026

14.1 Regressione

Processo per stimare una funzione data una quantità finita di elementi in un insieme rumoroso.

Si definisce una $h(x) = w_1x + w_0$, queste due costanti w **vanno trovate in modo sistematico**.

14.1.1 Regressione Lineare Univariata e Metodo Least Mean Square (LMS)

Si parte da una x in input e si ottiene una y in output secondo quindi la formula

$$\text{output} = h(x) = w_1x + w_0$$

Una volta definita la funzione di regressione questa può essere utilizzata per fare prediction, ma va costruita. Il metodo dei minimi quadrati può definire sistematicamente i due coefficienti w

Definizione LMS:

- Dato un insieme l di esempi di training del tipo (x_p, y_p)
- Si trova $h_w(x)$ nella forma $w_1x + w_0$ che minimizza la **loss** sui dati di training.
 - In particolare si trova w per minimizzare la somma dei quadrati:

$$\text{Loss}(h_w) = E(w) = \sum_{p=1}^l (y_p - h_w(x_p))^2 = \sum_{p=1}^l (y_p - (w_1x_p + w_0))^2$$

Utilizzo Gradiente: Per la minimizzazione di questa quantità, su una dimensione n si utilizza il gradiente empirico di $E(w) = 0$, ossia $\frac{\delta E(w)}{\delta w_i}$ per ogni $i = 1 \dots n$.

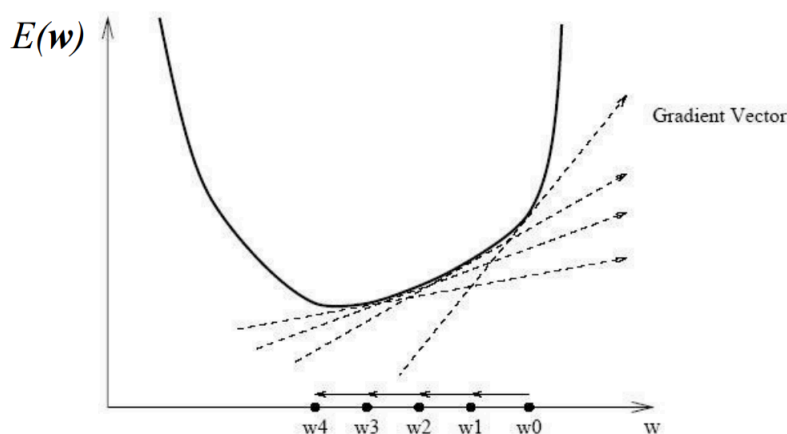
In Sintesi: Dato un insieme l di esempi di training (x_p, y_p) ed una funzione di loss, si trova il vettore peso w , in questo caso di dimensione 2, per la costruzione di una funzione $h_w(x)$ definita come $h_w(x) = w_1x + w_0$.

14.1.2 Approccio Iterativo via Ricerca Locale

Si basa sull'utilizzo del gradiente decrescente per la ricerca locale, quindi l'algoritmo iterativo definisce anche la dimensione di un passo ossia la step size η , quindi definendo:

$$\Delta w = - \text{gradiente di } E(w) \quad w_{\text{new}} = w + \eta \Delta w$$

Ricerca locale basata sull'assegnamento arbitrario della prima posizione, iterativamente verrà calcolato il passo successivo come definito da w_{new} sopra.



Si segue un criterio di error correction, detto anche delta rule, che cambia w in maniera proporzionale all'errore in base a questi casi:

- Se errore = $y_{\text{target}} - \text{output} = 0$ allora non viene effettuata **nessuna correzione**.
- Se output > target ossia $(y - h) < 0$ allora l'output è troppo alto:
 - Δw_0 negativa allora si riduce w_0 e
 - se input $x > 0$ con Δw_1 negativa allora si riduce w_1 .
- Se output < target ossia $(y - h) > 0$ allora l'output è troppo basso:
 - Δw_0 positiva allora si aumenta w_0 e
 - se input $x > 0$ con Δw_1 positiva allora si aumenta w_1 .

Proprietà del Gradiente Decrescente:

- Permette la ricerca locale in un infinito spazio d'ipotesi.
- Può essere applicata su spazio di ipotesi continue e funzioni differenziabili
- Non è il massimo d'efficienza, ma il concetto era quello di arrivare al minimo.

14.1.3 Estensione Pattern e Dimensioni Algoritmo a Gradiente Discendente

Elenchiamo passo passo cosa estendiamo:

- Abbiamo una **molteplicità di pattern**.
 - Questi possono essere sommati a piccoli passi, o a grandi batch.
- Pattern l viene **esteso** da una variabile indipendente ad una **dimensione** n .

Pattern	x_1	x_2	x_i	x_n
Pat 1	$x_{1,1}$	$x_{1,2}$		$x_{1,n}$
...				
Pat p	$x_{p,1}$	$x_{p,2}$	$x_{p,i}$	$x_{p,n}$
...				

- Possiamo quindi dereferenziare la tabella utilizzando l'indice di riga e di colonna.
- Secondo la notazione formale possiamo quindi scrivere:

$$w^T x = x^T w = w_0 + w_1 x_1 + \dots + w_n x_n = w_0 + \sum_{i=1}^n w_i x_i$$

- La funzione h sarà quindi definita come:

$$h(x_p) = x_p^T w = \sum_{i=0}^n x_{p,i} w_i$$

- Dove indichiamo il pattern con p e la feature con i .

In Sintesi:

- **Dato** un insieme l di esempi di training definiti come (x_p, y_p) dove
 - x_p è il vettore p -esimo, rappresentante il **pattern input**.
 - y_p è il **targer atteso corrispondente** al pattern p -esimo.
- **Trova** il vettore di pesi w che minimizza la loss attesa sui dati di training:

$$E(w) = \sum_{p=1}^l (y_p - x_p^T w)^2 = \|y - Xw\|^2$$

Algoritmo Generalizzato Gradiente Discendente:

- Si inizia con un vettore di pesi iniziale $(w)_{\text{initial}}$ piccolo, con una step size η compresa tra 0 ed 1.
- Calcola $\Delta w = -\nabla E(w) = -\frac{\delta E(w)}{\delta w}$ per ogni w_j .
- Calcola il nuovo passo come $w_{\text{new}} = w + \eta \Delta w$.
- Si ripete il secondo passo fino al raggiungimento della convergenza oppure ad un $E(w)$ sufficientemente piccolo.

14.2 Linear Basis Expansion (LBE)

Tutto quello mostrato fino ad ora tentava di modellare problemi lineari, ma alcuni problemi di regressione potrebbero richiedere approssimazioni non lineari.

Con lineare non s'intende per forza di cose una linea dritta, ma definisce in che modo i coefficienti di regressione in $\mathbf{w}$ si pongono nell'equazione di regressione $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$. L'importante è che i pesi nel vettore $\mathbf{w}$ restino lineari. Volendo spiegare informalmente la LBE:

- I pesi $(w_1, \dots, w_n)$ restano “puliti”, quindi lineari.
- Le variabili (feature) contenute nel vettore $\mathbf{x} \in \mathbb{R}^n$ vengono passate ad una funzione $\Phi(\mathbf{x}) \in \mathbb{R}^K$, con n numero di feature originale e K numero di nuove variabili costruite.
- Questo aumenta il numero di nuove variabili, ma permette di modellare il problema negli stessi termini definiti prima per problemi lineari, basta che i pesi in $\mathbf{w}$ restino lineari.

Si possono anche usare input $\mathbf{x}$ trasformati con **relazioni non lineari** per riferire nuovamente a soluzioni come quella della LMS.

Definizione: Definiamo quindi la trasformazione Linear Basis Expansion:

$$h_{\mathbf{w}}(\mathbf{x}) = \sum_{k=0}^K w_k \Phi_k(\mathbf{x})$$

- $\Phi_k : \mathbb{R}^n \rightarrow \mathbb{R}$, quindi funzione che permette la trasformazione di $\mathbf{x}$, ad esempio:
 - **Rappresentazione polinomiale:**

$$\Phi(\mathbf{x}) = x_j^2 \quad \Phi(\mathbf{x}) = x_j x_i$$

- **Trasformazione non lineare a singolo input:**

$$\Phi(\mathbf{x}) = \log(x_j) \quad \Phi(\mathbf{x}) = \sqrt{x_j}$$

- **Trasformazione non lineare multi input:**

$$\Phi(\mathbf{x}) = \|\mathbf{x}\|$$

Caratteristiche LBE: Questo approccio dipende pesantemente da quale Φ si sceglie per la trasformazione.

- **Pro:** Grazie all'utilizzo di questo approccio possiamo modellare relazioni più complesse di quelle lineari.
- **Contro:** Ci servono metodi per tenere sotto controllo la complessità del modello, gestendo eventuali fenomeni di **underfitting** ed **overfitting**.

15 — — — — — Lezione 12 - Modelli Lineari II - 24/03/2026

15.1 Underfitting/Overfitting e Regularizzazione

La funzione generata dipende dalla **complessità del modello**, ossia la sua **flessibilità** sul fare **fitting sui dati**.

Regularizzazione: Ci permette di controllare fenomeni di overfitting penalizzando funzioni complesse effettuando uno shrink dei pesi a queste funzioni.

Un errore di training molto basso non garantisce nulla da un punto di vista di buona generalizzazione. Per questo motivo si preferisce agire sul vettore di pesi con metodologie di normalizzazione.

15.1.1 Regularizzazione Ridge/Tikhonov

Si vincola sui valori di w_i , favorendo modelli sparsi con meno termini a causa dei pesi $w_j = 0$. Effettua quindi un'operazione di smoothing, arrivando a modelli meno complessi.

$$\text{Loss}(h_w) = \sum_{p=1}^l (y_p - h_w(x_p))^2 + \lambda \|w\|^2$$

Definito ad alto livello quindi sarebbe la **somma** tra l'**errore** ed il **termine di regolarizzazione**, dove $\|w\|^2 = \sum_i w_i^2$.

L'effetto sul nuovo vettore di pesi calcolato sarà quindi:

$$w_{\text{new}} = w + \eta \Delta w - 2\lambda w$$

Fino ad ora la definizione del polinomio era piena definizione di bias di linguaggio. Questo meccanismo di preferenze introdotto tramite l'utilizzo del λ tocca invece il concetto di bias di ricerca, perchè stiamo preferendo alcune ipotesi rispetto ad altre.

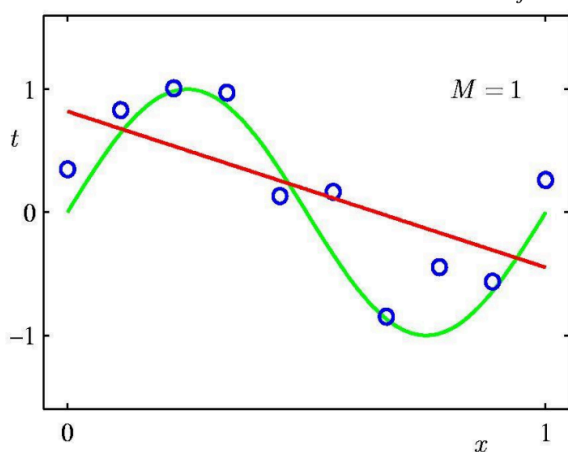


Figure 32: Esempio di Underfitting

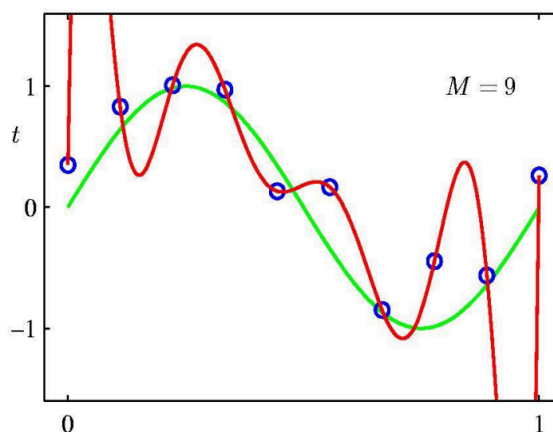
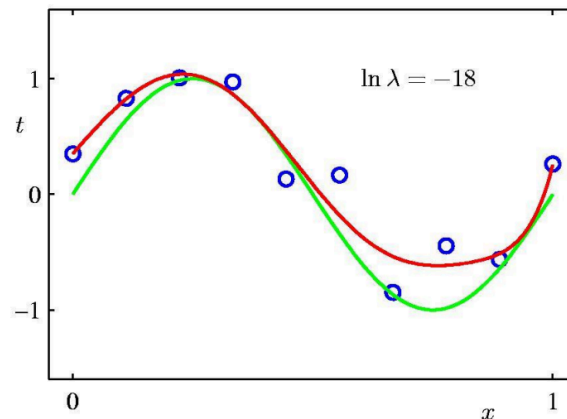


Figure 33: Esempio di Overfitting

Rappresentazione Funzione Regularizzata: Una funzione regolarizzata con $\ln \lambda = -18$ appare così:



15.2 Classificazione per Regressione

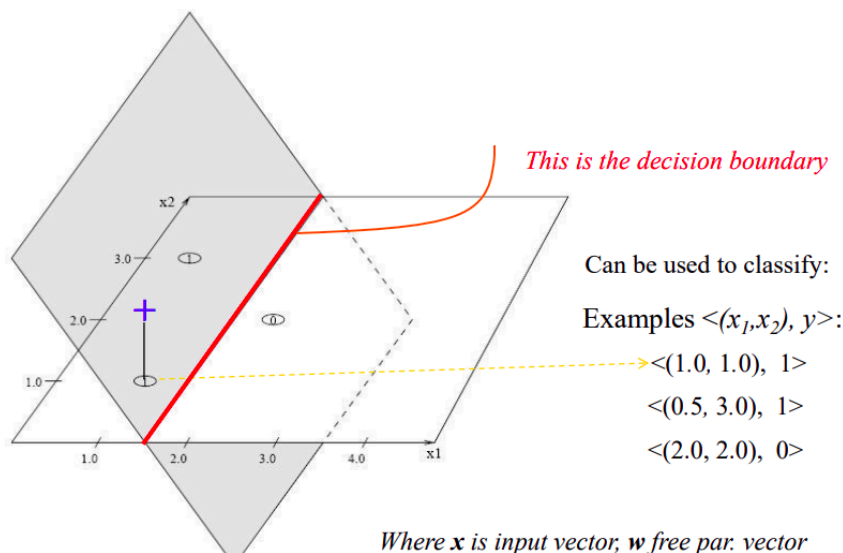
Riutilizziamo la classificazione della regressione per definire se un oggetto appartenga o meno alla parte positiva o negativa di un iperpiano $x^T x$.

L'idea quindi non è utilizzare la funzione ipotizzata per stimare un valore numerico, ma classificare un valore $+1$ true oppure un -1 false.

15.2.1 Decision Boundary

Informalmente dove l'iperpiano vale 0, quindi formalmente:

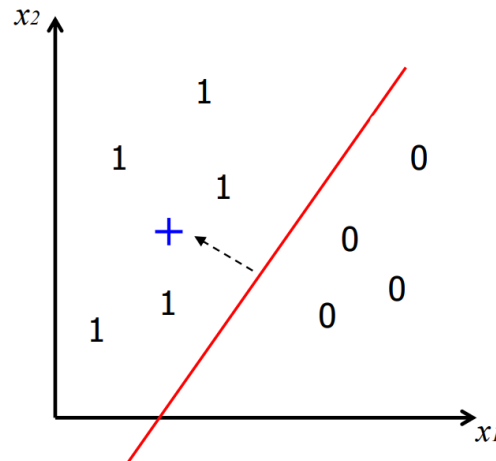
$$x^T x = w_1 x_1 + w_2 x_2 + w_0 = 0$$



15.2.2 Decision Boundary Threshold Classifier

La classificazione può essere vista come la divisione dello spazio degli input in regioni di decisione, nel caso binario due regioni $\{0, 1\}$.

Nel caso semplice a due dimensioni, la funzione determina la soglia tra le due regioni.



Si definisce quindi:

$$h(x) = \text{sign}(w x + w_0) \quad h(x) = \begin{cases} 1 & \text{if } w x + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$h(x_p) = \text{sign}(x_p^T w) = \sum_{i=0}^n x_{p,i} w_i$$

Questa è detta **Linear Threshold Unit (LTU)**.

Il vantaggio quindi è quello di poter utilizzare gli stessi strumenti di prima (LBE e Regolarizzazione) per una classe di problemi differenti, ossia quelli di classificazione.

15.2.3 Formalizzazione Learning Problem for Linear Classifiers

Assumendo di utilizzare ancora il Least Min Square:

- Sia **dato** un insieme di esempi di training l .
- Si **trova** il vettore w che minimizza la somma residua dei quadrati:

$$E(w) = \sum_{p=1}^l (y_p - x_p^T w)^2 = \|y + Xw\|^2$$

Attenzione: A differenza della regressione non utilizziamo in $E(w)$ la forma $h(x)$ perchè se definita a tratti potrebbe risultare non differenziabile. Di conseguenza $h(x) = \text{sign}(w^T x)$ viene usato solo per la classificazione.

Questo ci permette l'utilizzo **anche in questo contesto** dell'**algoritmo iterativo a gradiente discendente**.

Riassumendo:

- I **modelli** vengono **allenati** su training set tramite l'utilizzo del Least Min Square su $w^T x$. Nello specifico si allena utilizzando l'**algoritmo del gradiente discendente** per la **regressione lineare**.
- I **modelli** possono anche **essere utilizzati** però per la **classificazione** utilizzando la funzione come threshold, ottenendo quindi analiticamente $h(x) = \text{sign}(w^T x)$.
 - In questo caso di utilizzo è possibile quindi calcolare una **accuracy** e complementare percentuale di errore di classificazione secondo il calcolo:

$$\text{accuracy} = \frac{l - \text{num_err}}{l} = 1 - \text{mean_err}$$

16 — — — — — Lezione 13 - Decision Trees - 26/03/2026

Il **Decision Tree** è una **visualizzazione grafica di espressioni composte** da $\vee, \wedge$.

16.1 Algoritmo ID3

L'algoritmo **ID3** è pensato per l'apprendimento basato su Decision Tree, in particolare per la costruzione in **top down greedy** dell'albero.

Descrizione in Pseudocodice:

- Dato un training set di esempi, l'algoritmo per la costruzione del DT effettua una ricerca nello spazio degli alberi DT.
- La **costruzione** è definita in **top down**, l'algoritmo effettua una **ricerca greedy**.
- La **domanda fondamentale** è "quale è il prossimo attributo da testare?"
 - Si seleziona l'**attributo migliore**, secondo le funzioni Entropy e Gain definite dopo.
 - Un **nodo discendente** è creato per **ogni possibile valore** di questo **attributo migliore**, e gli **esempi** sono anch'essi **partizionati** secondo questo **attributo migliore**.
 - Questo **processo** viene **reiterato fino a quando** tutti gli **esempi** sono **correttamente classificati** o fino a quando **non sono più presenti attributi**.

Oltre alla definizione in se dell'algoritmo ricorsivo, bisogna analizzare cosa si intenda per **scelta migliore**. Abbiamo quindi la necessità di definire cosa sia l'**entropia**.

16.1.1 Definizione Entropia

L'entropia misura quanto impura sia una collezione di esempi, viene calcolata come:

$$0 \leq \text{Entropy}(S) \equiv -p_+ \log_2(p_+) - p_- \log_2(p_-) \leq 1$$

Questo quindi ci permette di ripartire la collezione, in questo contesto siamo quindi interessati agli estremi (0, 1).

16.1.2 Definizione Gain

Il gain è la riduzione attesa in entropia causata dal partizionamento dell'insieme su un attributo.

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

L'**obiettivo** è quello di scegliere A in maniera tale da **massimizzare** il **gain**, quindi **diminuendo** il **termine a destra della sottrazione**.

Nello specifico il termine a destra della sottrazione corrisponde, per ogni sottoalbero, all'entropia di ciascun ramo in questione. Quindi **viene scelto** quello ad **entropia inferiore**.

- Da un **punto di vista semantico**, stiamo classificando meglio a **gain alti**, quindi stiamo **differentiando meglio gli elementi** nei sottoinsiemi.

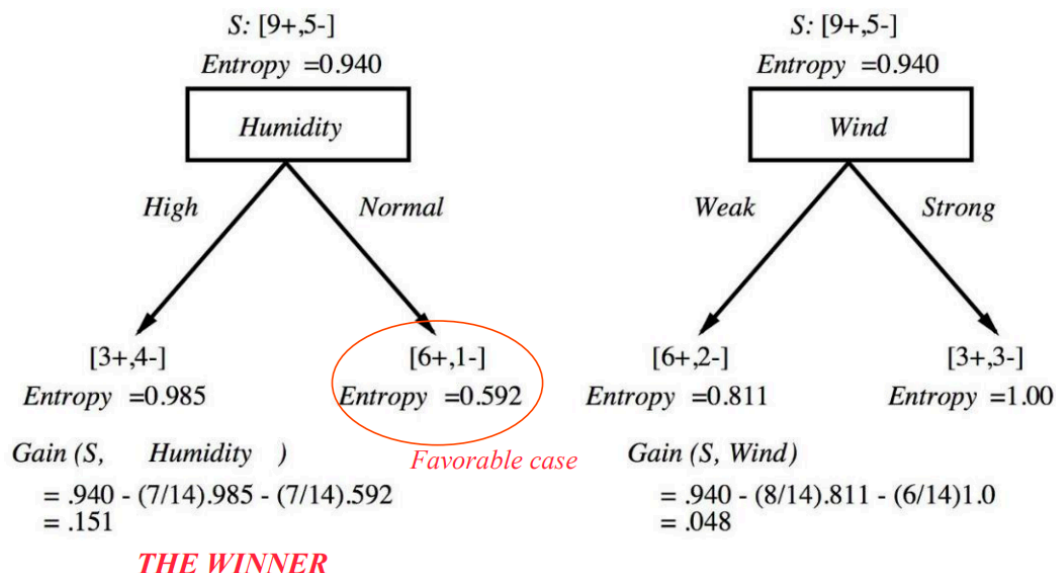


Figure 37: Nota che per ogni branch cambia solo il valore del termine sinistro della sottrazione.

Correttivo al Information Gain: Vogliamo evitare la polverizzazione in numerosità di classificazioni, quindi non vogliamo creare troppi branch. Si definisce quindi un GainRatio().

$$\text{GainRatio}(S, A) \equiv \frac{\text{Gain}(S, A)}{\text{SplitInformation}(S, A)}$$

dove

$$\text{SplitInformation}(S, A) \equiv - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

Questo **semanticamente** indica la **penalizzazione di casi** che portano ad **alto branching**.

Ulteriore Correttivo: Il valore di SplitInformation(S, A) può diventare 0 o un valore molto piccolo quando |S_i| è circa quanto |S|.

- Per risolvere questa problematica si applicano euristiche per cui:
 - Si calcola la Gain per ogni attributo.
 - Si applica la GainRatio su attributi con la Gain sopra la media, applicando quindi questo GainRatio solo quando necessario.

16.2 Inductive Bias del Learning su Decision Tree (DT)

I bias induttivi del DT learning:

- **Alberi corti** sono **preferiti** ad **alberi lunghi**.
- Si **preferiscono** alberi che piazzano informazioni con attributi ad **alto gain vicino alla radice**.

16.3 Bias di Ricerca vs Bias di Linguaggio in Modelli Discreti/Lineari

- **Modelli Discreti:**
 - Definite **preferenze** con **Bias di Ricerca**, causa **strategia di ricerca**:
 - L'**algoritmo ID3** cerca uno **spazio di ipotesi completo**, mentre la **strategia di ricerca è incompleta**.
 - Definite **restrizioni** con **Bias di Linguaggio**, causa insieme di ipotesi esprimibile o considerato:
 - L'**algoritmo Candidate-Elimination** cerca uno **spazio delle ipotesi incompleto**, mentre la **strategia di ricerca è completa**.
- **Modelli Lineari:**
 - Si utilizzano **combinazioni dei due bias**, con **preferenza su quello di linguaggio** per **flessibilità**, stando sempre attenti al fenomeno dell'Overfitting.

16.4 Definizione di Overfitting

Data un ipotesi h in uno spazio di ipotesi H :

- Si considerano due errori:
 - Su **training set** $\text{error}_D(h)$, quindi **errore empirico**.
 - Sull'**intera distribuzione** $\text{error}_X(h)$, quindi **errore reale**, permette di stimare la **bontà della generalizzazione**.

Definizione: Un ipotesi $h \in H$ fa **overfitting sui dati di training** se esiste un **ipotesi alternativa** $h' \in H$ tale che

$$\text{error}_D(h) < \text{error}_D(h') \quad e \quad \text{error}_X(h') < \text{error}_X(h)$$

Quindi informalmente, h' **alternativa fitta peggio sui dati di training** ma **generalizza meglio sui nuovi esempi**.

16.4.1 Strategie di Mitigazione Overfitting (Early Stopping, Pruning)

Esistono diverse strategie:

- **Early Stopping:** Fermarsi prima nello sviluppo dell'albero, prima della classificazione piena.
- **Pruning:** Permettere l'overfit e successivamente effettuare operazioni di **prune** tagliando rami dell'albero.
 - **Reduced-Error Pruning:**
 - Rimozione di interi sottoalberi di nodi, effettuata iterativamente.
 - I nodi vengono rimossi solo se gli alberi risultanti non performano peggio di quelli precedenti sul validation set e per ciascuna iterazione vengono rimossi i nodi la cui rimozione apporta un maggior incremento di accuracy.
 - La reiterazione si ferma quando nessun altro pruning porta a miglioramenti sull'accuracy

- **Rule Post-Pruning:**
 - Si **parte** da un **DT**.
 - Si **converte in DT** in un **insieme di regole equivalente**, dove:
 - Ogni **path** mappa su una **regola**.
 - Ogni **nodo** mappa su una **pre-condizione**.
 - Ogni **foglia** mappa su una **post-condizione**.
 - Si effettua **pruning** sulle **pre-condizioni** la cui **rimozione migliora l'accuracy** sul validation set oppure sui dati di training.
 - Si **ordinano le regole** su una **stima di accuracy**, considerandole in sequenza durante la classificazione di nuove istanze.

16.5 Gestione Valori Continui, Gestione Valori Mancanti e Gestione Attributi a Costi Non Omogenei

Gestione Valori Continui: Possiamo utilizzare un approccio discreto anche su valori continui, utilizzando i valori discreti come soglie.

Si determinano le soglie stabilendo dove mediamente esiste un cambio di classificazione tra i valori continui ordinati.

Gestione Valori Mancanti: In contesti reali è molto probabile che alcuni attributi non vengano istanziati, di conseguenza si stabiliscono delle strategie per la gestione di queste casistiche:

- Si sostituiscono i campi non compilati con il valore più comune della stessa classe.
- Si gestisce in maniera più a grana fine utilizzando una distribuzione ed associando una probabilità p per ciascun valore v .

Gestione Attributi a Costi Non Omogenei: Gli attributi delle istanze potrebbero avere un costo associato. Si preferiscono DT con attributi a basso valore. In questo modo si definiscono modifiche di ID3 per tenere conto dei costi:

- **Tan and Shmmler:**

$$\frac{\text{Gain}^2(S, A)}{\text{Cost}(A)}$$

- **Nunez:**

$$\frac{2^{\text{Gain}^2(S, A)} - 1}{(\text{Cost}(A) + 1)^w} \quad \text{con } w \in [0, 1]$$

17 — — — — Lezione 14 - Validation and Theoretical Issues - 31/03/2026

17.1 Rapida Sintesi su Fasi Learning/Prediction e Capacità di Generalizzazione

Si torna sulla **generalizzazione**, abbiamo visto anche criteri di regolarizzazione sia nel continuo sia nel discreto.

Ricapitolando cosa abbiamo visto nelle precedenti lezioni di ML avevamo due fasi principali:

- **Fase di Learning: Costruzione del modello** partendo dai dati conosciuti, quindi dai **dati di training e dai bias**.
- **Fase di Prediction (Test)**: Si acquisiscono nuovi input x' e si genera la risposta del modello. Questa risposta viene confrontata con il suo target atteso d' che il modello non ha mai visto.
 - Sulla base di questo di questo **si stabilisce la capacità di generalizzazione di un modello**.

17.2 Model Selection vs Model Assessment

Si definiscono due fasi fondamentali della validazione:

1. **Model Selection**: Si stima la performance, sulla base dell'errore di generalizzazione, di modelli diversi al fine di scegliere quello che generalizza meglio.
 - Quindi si sceglie il modello, gli iperparametri.
 - Questa **fase ritorna un modello finale**, quello migliore.
2. **Model Assessment**: Avendo scelto il modello finale, bisogna stimare come si comporterà il modello in futuro in termini di generalizzazione su nuovi dati di test, per misurare la sua capacità di generalizzazione.
 - Questa **fase ritorna una stima sul modello finale**.

17.3 Validazione su Stima Empirica

17.3.1 Hold Out - Ripartizione del Set

Approccio per cui si ripartisce l'**insieme di dati complessivo** in **sottoinsiemi disgiunti**, quindi:

$$\text{Set} = \text{Training Set} \cup \text{Validation Set} \cup \text{Test Set}$$

dove

$$\text{Development/Design Set} = \text{Training Set} \cup \text{Validation Set}$$

Il principio per cui i **sottoinsiemi** devono essere **disgiunti** è fondamentale, costruire un modello sulla base del Test Set causerà una **finta buona stima** in fase di Assessment. Questo porterà ad una **stima sovraottimistica**. In sintesi:

- Il Training Set va usato nella **fase di Training**.
- Il Validation Set va usato per la **Model Selection**.
- Il Test Set va usato per la **Risk Estimation**.

17.3.2 Pseudoalgoritmo Grid Search - Hold Out

Definizione dell'algoritmo che permette l'utilizzo a partizioni del dataset:

1. Si separa l'intero dataset in Training Set TR, Validation Set TL e Test Set TS.
2. Si cerca il **miglior modello** $h_{w,\lambda}$ **cambiando gli iperparametri** λ , che possono ad esempio essere l'ordine dei polinomi, oppure il lambda per la normalizzazione.
3. **FOR ESTERNO**: Per ogni valore diverso di λ :
 - 3.1. **FOR INTERNO**: Si cerca la migliore $h_{w,\lambda}$ che minimizza l'errore empirico, quindi che fitta meglio il TR set, trovando il w di parametri migliore. Quindi con migliore in questo caso si intende

$$\operatorname{argmin}_w \operatorname{Loss}(w) \in L_2$$

4. Opzionale: Si può, dopo le esecuzione dei FOR, far fittare la $h_{w,\lambda}$ su TR Set + VL Set.
5. In conclusione si valuta la finale $h_{w,\lambda}(x)$ sul TS Set.

17.3.3 Potenziali Errori causati da Cattiva Suddivisione dataset

- **Stima Rischio, TR e VL**: L'errore che stimiamo su training o validation nel selection model non è utile per la stima del rischio. I dati di Training e di Validation non vanno mai usati per scopi di test.
- **Feature Selection Bias**¹: Non va usato tutto il dataset durante la selezione delle feature o del modello, perchè questo può indurre ad una specifica scelta rispetto ai risultati che utilizzeremo per i test, quando non dovremmo mai essere a conoscenza di quei dati durante allenamento e validazione.
 - In questo senso stiamo quindi **mettendo in discussione la correttezza della stima** e non la possibilità di risolvere il task.

17.3.4 K-Fold Cross Validation

Tecnica per cui si ottimizza l'utilizzo del dataset, partizionando in maniera "dinamica" quest'ultimo invece che staticamente.



Definizione Si seguono questi passi:

- Si splitta il dataset D in k sottoinsiemi mutualmente esclusivi.
- Si allena l'algoritmo su D/D_i e si testa su D_i .
- Si tiene una media dei risultati dati dalle D_i della diagonale.

Questo permette un utilizzo più efficiente dei dati a scambio di maggiore complessità computazionale.

¹In questo caso bias con accezione negativa.

17.4 Validazione - Fondamenti Teorici tramite Statistical Learning Theory - SLT

Teniamo conto e motiviamo con teoria formale elementi già citati precedentemente:

- **Capacità di generalizzazione**, misurata in termini di Risk o Test Error di un modello.
 - Tenendo conto dell'errore di training.
 - Gestendo zone di **overfitting** ed **underfitting**.
- Ruolo della **complessità del modello**, misurata in termini di VC-dim.
- Ruolo della **quantità di dati** misurati in l .

17.4.1 Setting Formale della Statistical Learning Theory (SLT)

1. Si vuole approssimare una funzione $f(x)$ reale con d che corrisponde al **target**,

quindi $d = \text{true } f + \text{noise}$

2. Minimizzare la funzione Risk:

$$\text{Risk} = R = \int L(d, h(x)) dP(x, d)$$

3. Siano dati

- Un valore da d e la distribuzione di probabilità $P(x, d)$
- Una funzione Loss definita come:

$$L(h(x, d)) = (d - h(x))^2$$

4. Si cerca h ipotesi in H tale per cui si minimizza Risk.
5. Ma i dati nel nostro dataset non sono infiniti, infatti $\text{TR} = (x_p, d_p)$ con $p = 1 \dots l$.
6. La h quindi si cerca minimizzando un Risk empirico, Risk_{emp} , definito come:

$$R_{\text{emp}} = \frac{1}{l} \sum_{p=1}^l (d_p - h(x_p))^2$$

- Non diamo per scontato che il Risk_{emp} approssimi bene l'originale Risk, lo argomentiamo tramite la teoria di *Vapnik-Chervonenkis*.

17.4.2 Teoria di Vapnik-Chervonenkis

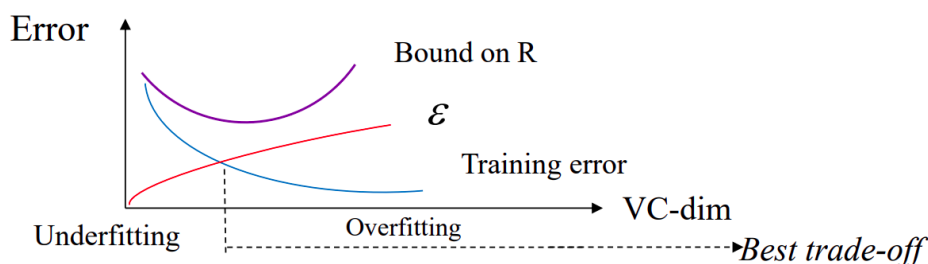
- Data la VC-dim (VC), ossia una misura di complessità di H , quindi la flessibilità nel fittare i dati del TS.
- Settiamo un bound sul Risk nella seguente forma:

$$R \leq R_{\text{emp}} + \varepsilon \left(\frac{1}{l}, \text{VC}, \frac{1}{\delta} \right) \quad \text{con} \quad \varepsilon \left(\frac{1}{l}, \text{VC}, \frac{1}{\delta} \right) = \text{VC-confidence}$$

- δ è la confidence, regola la probabilità che mantiene il bound, mentre l è la quantità di dati.
- ε è una **funzione** che **cresce al crescere** di VC e **decresce al crescere** di l e δ .
- Sappiamo che R_{emp} decresce utilizzando modelli complessi, quindi ad alta VC.
- Grazie a questa formulazione possiamo anche spiegare i fenomeni di Overfitting ed Underfitting:
 - **Underfitting**: Modello poco complesso a bassa VC non sufficiente data l'alta R_{emp}

- **Overfitting:** Modello molto complesso ad alta VC su un numero limitato l di dati può abbassare R_{emp} ma la VC-confidence aumenta, aumentando R .

Def. Minimizzazione del Risk Strutturale Si tenta di minimizzare il bound settato.



Questo definisce il controllo sulla complessità (flessibilità) di un modello, definendo quindi un trade-off tra fitting sul TR e complessità effettiva del modello in termini di VC.

18 — — — — Lezione 15 - Support Vector Machine (SVM) - 14/04/2026

18.1 Definizione SVM e Obiettivi

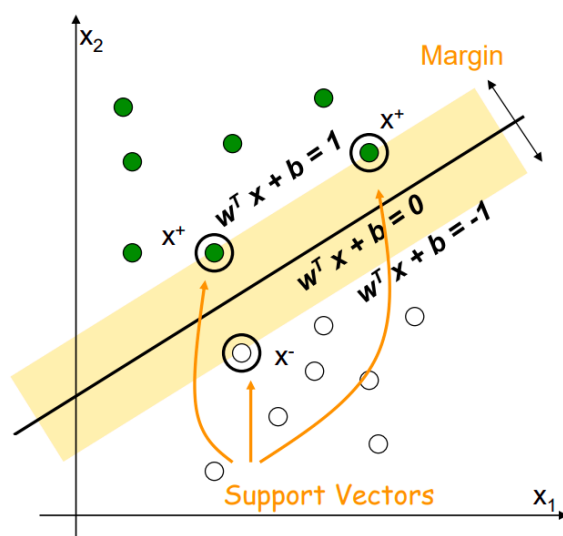
Obiettivi SVM: Modello della SVM argomento di questa lezione, basato su tre obiettivi:

1. Controllare la complessità del modello tramite un approccio di ottimizzazione, tramite l'utilizzo di un **Max Margin Classifier**
2. Utilizzo della **Linear Base Expansion** via **kernel**
3. Si **evitano** condizioni di **overfitting**, dando dei criteri di fiducia.

18.1.1 Obiettivo 1: Maximum Margin Classifier

Si assume un problema che sia linearmente separabile.

- Non tutti modelli sono equivalenti, perchè se definiamo un margine questo varia tra soluzioni valide.



Dove si definiscono i support vectors come $x_p = |w^T x_p + b| = 1$

Quindi considerando il problema di definizione di un modello lineare per classificazione binaria ossia $h(\mathbf{x}) = \text{sign}(\mathbf{x}\mathbf{w} + b)$.

Def. Training Problem: Cercare $(\mathbf{w}, b)$ tale per cui tutti i **punti sono classificati correttamente** ed il **margin** è **massimizzato**:

$$(\mathbf{w}^T \mathbf{x}_p + b)y_p \geq 1$$

18.1.1.1 Rapporto VC-DIM e Massimizzazione Margine

- La **massimizzazione** del **margin** causa **minimizzazione** di $\|\mathbf{w}\|$.
- La **VC-dim della SVM** ha un **valore inverso** a quello del **margin**, di conseguenza l'**iperpiano ottimale** è quello che **massimizza il margin**.

18.1.1.2 Hard Margin SVM: Problema di Ottimizzazione Quadratica

Questo problema è definito come **primale** su ottimizzazione quadratica secondo queste formulazioni:

- **Def. di Training Problem:** Cercare $(\mathbf{w}, b)$ tale per cui tutti i **punti sono classificati correttamente** ed il **margin** è **massimizzato**:
- **Def. di Training Problem in Forma Primale:** Minimizzare $\frac{\|\mathbf{x}\|^2}{2}$ tale per cui

$$(\mathbf{w}^T \mathbf{x}_p + b)y_p \geq 1 \text{ per ogni } p = 1 \dots l$$

.

Formulazione Duale: Solitamente non si risolve tramite il duale ma torna comoda la sua formulazione. Questo perchè il duale non si basa sui pesi dei $\mathbf{w}$ ma definisce h sulla base dei soli vettori di supporto.

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b) = \text{sign}\left(\sum_{p=1}^l \alpha_p y_p \mathbf{x}_p^T \mathbf{x} + b\right) = \text{sign}\left(\sum_{p \in \text{SV}} \alpha_p y_p \mathbf{x}_p^T \mathbf{x} + b\right)$$

18.1.1.3 Soft Margin SVM

Fino ad ora abbiamo **ipotizzato un ambiente ideale senza iperparametri** forniti dall'utente. Nella soft margin invece includiamo l'utilizzo di un iperparametro $C > 0$ che guida il **numero di errori ammessi**.

Primal Training Problem: Minimizzare $\frac{\|\mathbf{w}\|^2}{2} + C \sum_p \xi_p$ quindi:

- **Basso Valore C :** Più errori ammessi sul TR, possibile underfitting.
- **Alto Valore C :** No errori ammessi sul TR, possibile overfitting.

18.1.2 Obiettivo 2: Kernel Trick in Linear Basis Expansion

Dato che uno dei **possibili problemi** era avere **spazi non divisibili linearmente** si utilizza una funzione di mapping Φ , tramite LBE già vista, che parte dalla dimensione originale ad una più grande.

L'idea in questo contesto è però quella di normalizzare **non utilizzando** direttamente il prodotto scalare $\Phi^T(\mathbf{x}_p)\Phi(\mathbf{x})$ contenuto in

$$h(x) = \text{sign}(\alpha_p y_p \Phi^T(\mathbf{x}_p)\Phi(\mathbf{x}))$$

ma utilizzando $\Phi^T(\mathbf{x}_p)\Phi(\mathbf{x}) = K(\mathbf{x}_p, \mathbf{x})$ quindi:

$$h(x) = \text{sign}(\alpha_p y_p K(\mathbf{x}_p, \mathbf{x}))$$

18.1.2.1 Kernel Noti

Esistono noti Kernel a cui possiamo fare riferimento:

- **Lineare:**

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$$

- **Polynomial:**

$$K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^k$$

- **Radial Basis Function - (RBF) Gaussian:**

$$K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\frac{(\|\mathbf{x}_i - \mathbf{x}_j\|)^2}{2\rho^2}}$$

18.1.2.2 Formulazione Finale SVM

Riassumendo:

- Scelta iperparametro C e funzione kernel K .
- Si risolve il problema di ottimizzazione per trovare α

Il modello finale è quindi:

$$h(x) = \text{sign}(\alpha_p y_p K(\mathbf{x}_p, \mathbf{x}))$$

18.1.3 Obiettivo 3: Evitare Interpretazioni Sbagliate

Esistono tipiche interpretazioni sbagliate dell'SVM:

- **Overfitting:** Può occorrere a causa della **scelta sbagliata di iperparametri**.
- Le **tecniche di validazione** andrebbero utilizzate rigidamente per fare selezione del modello corretto.

19 — — — — — Lezione 16 - K-NN, Unsupervised Learning e Altri Approcci - 16/04/2026

Lezione basata su paradigmi lasciati un po' da parte in tutte le altre lezioni.

19.1 K-Nearest Neighbors (Supervised Learning)

Si basa su questa definizione:

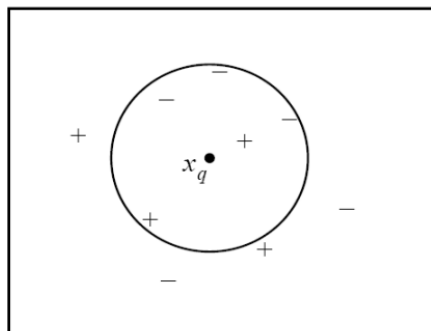
- Date le coppie (x_p, y_p)
- Dato un input x con dimensione n .
- Trova l'esempio x_i nel training set che è più vicino all'input, sulla base di una distanza euclidea:

$$i(x) = \arg \min d(x, x_p) \quad \text{dove} \quad d(x, x_p) = \sqrt{\sum_{t=1}^n (x_t - x_{p,t})^2} = \|x - x_p\|$$

Questo permette di l'output di y_i .

In questo modo definiamo un 1-NN, quindi un approccio secco sul dato più vicino nel TR.

Smoothing NN: Se definiamo un insieme di k più vicini abbiamo modo di ammorbidire la scelta in base ad una **media**.



Motivazione Supervisionato: Questo algoritmo è supervisionato perchè stiamo guardando la coordinata y_i nel TS quindi abbiamo un etichetta di riferimento.

19.1.1 Definizione Formale K-Nearest Neighbors

Un modo naturale quindi per classificare un nuovo punto è quello di guardare i suoi vicini e farne una media $\text{avg}_k(x) = \frac{1}{k} \sum_{x_i \in N_k(x)} y_i$.

19.1.2 Caratteristiche K-Nearest Neighbors

Si basa sulla memorizzazione del TR, ma non definisce un effettivo modello ma utilizza a tempo debito l'accesso al TR.

- Si elimina la fase di training, ma la fase di predizione è molto più consistente.
- Per questo motivo si definisce come **lazy**, **memory based**, **instance based** e **distance based**.
- Si perde la compattezza del modello lineare e costa molto da un punto di vista computazionale (spazio e tempo).

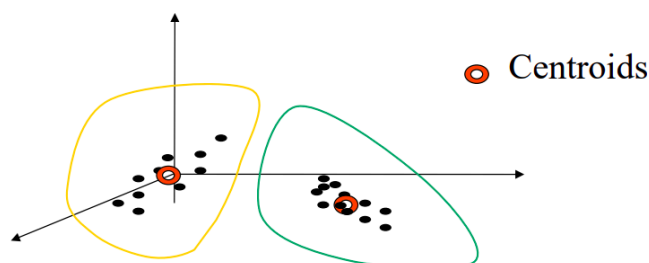
- A causa della memorizzazione, il volume a crescita esponenziale impatta negativamente sulla densità dei dati.
- Quindi questo funziona fino a quando i dati sono realmente vicini, ma questo all'aumentare delle dimensioni risulta essere molto fragile.
- La scelta della metrica è l'inductive bias in questo caso, dato che la scelta della metrica non è fissata.
- Il trade-off tra overfitting e underfitting è gestito dall'iperparametro K .
 - Un esempio estremamente flessibile con $K = 1$.
 - Un esempio rigido con $K = l$.

19.2 Approcci di Unsupervised Learning

Si mostrano approcci di Unsupervised Learning.

19.2.1 K-Means (Unsupervised Learning)

Si assume un TR **senza label target** quindi dati della forma (x) .



- Classico per riconoscimento naturale a cluster di dati.
- Non si definiscono solo **cluster (pattern)** ma anche **centroidi**, quindi rappresentanti di quel pattern.
 - Si discretizza **mappando** quindi l'intero cluster sul centroide, minimizzando una loss, ossia distanza tra il pattern ed il suo centroide:

$$L(h(x_p)) = \|x_p - c(x_p)\|^2$$

19.2.1.1 Basic K-Means Batch Alg. - (LBG, Generalized Lloyd Alg.)

Def. Algoritmo di Ricerca Locale:

- Scegli k centroidi a caso, posizionati in maniera randomica all'interno dello stesso spazio dei pattern.
- Assegna ad ogni pattern al relativo centroide più vicino.
- Ricalcola i centroidi rispetto ai membri dei cluster.
- Se non si raggiunge il criterio di convergenza si ritorna al secondo step.

Dettagli Algebrici: Dati i centroidi $c_1, \dots, c_k$, per ogni vettore x il vincitore è il prototipo più vicino:

- $i^*(x) = \arg \min \|x - c_i\|^2$
- Per ogni cluster indicizzato con i il nuovo **centroide basato su media** è:

$$c_i = \frac{1}{|\text{cluster}_i|} \sum_{j: x_j \in \text{cluster}_i} x_j$$

19.2.1.2 Caratteristiche K-Means

- Il numero di cluster deve essere definito a priori.
- La minimizzazione della loss $L(x)$ dipende pesantemente dalla scelta iniziale, dato che la sua minimizzazione segue criteri di discesa locale.
- Non ha proprietà di visualizzazione in caso di alto numero di dimensioni.

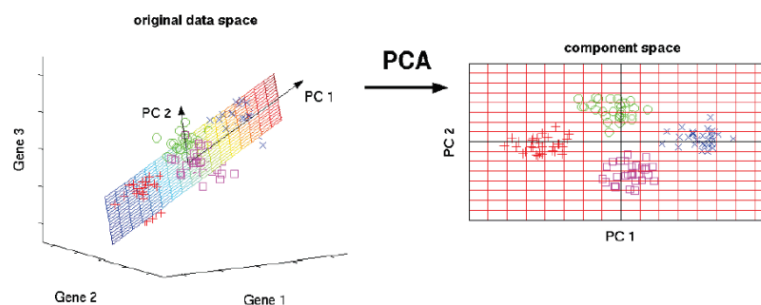
Valutazione dei Cluster Identificati: Come si definisce se sia stato effettuato o meno un buon clustering?

- L'evalutazione può essere potenzialmente soggettiva, non esistono a priori standard di riferimento.
- Esistono però metriche di purezza, errore di quantizzazione o entropia per definire quanto un cluster si allontani da una classe di oggetti presistenti.

19.2.2 Dimensionality Reduction via Principal Component Analysis (PCA)

Grazie a questo metodo possiamo ridurre un numero di feature in dimensione n su una combinazione di feature in dimensione n' con $n > n'$.

- Si definiscono assi nello spazio a maggior dimensioni che vengono usati per la generazione dello spazio a dimensione inferiore.



19.3 Reinforcement Learning e Semi-Supervised Learning

Non **Supervised** ma nemmeno **Non Supervised**, tramite feedback all'agente di **rewards** e **punishment** non sull'azione atomica ma su tutto il comportamento che sta assumendo.

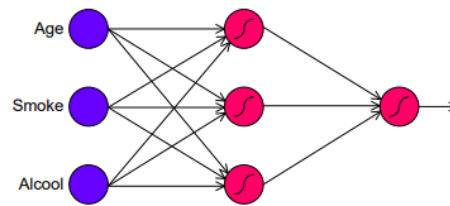
- Utilizzato in contesti come grafi, alberi, vettori di pattern...

Esiste anche il **Semi-Supervised Learning** per cui si combinano esempi etichettati con esempi non etichettati, per generare una funzione o un classificatore appropriato.

19.4 Neural Networks

Possono essere sia Supervised sia Non Supervised, ma in questa lezione si trattano quelle Supervised.

- **Perceptron:** Unità base della rete neurale.



- Si basano su **gestione di errori** con **backpropagation** e approccio a **gradiente discendente**.
- I layer successivi al primo sono gestiti con pesi w adattivi gestiti con **basis expansion** dove le Φ sono imparate dal modello progressivamente.
- **Deep Learning:** Layer molteplici di unità di calcolo, tutti i livelli successivi al primo sono caratterizzati da un **livello di astrazione crescente**.

